

Performance Evaluation of Machine Learning Models for Intrusion Detection in MQTT-based IoT Environments

by

MD. MUSFIQUE BILLAH

20101002

HRIDITA BARUA

20101003

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
January 2026.

© 2026. Brac University
All rights reserved.

Declaration

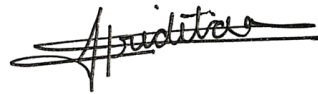
It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:



Md. Musfique Billah
20101002



Hridita Barua
20101003

Approval

The thesis titled “ Performance Evaluation of Machine Learning Models for Intrusion Detection in MQTT-based IoT Environments ” submitted by

1. Md. Musfique Billah (20101002)
2. Hridita Barua (20101003)

of Fall, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science on 31 January, 2026.

Examining Committee:

Supervisor:
(Member)



Dr. Md. Iqbal Hossain

Associate Professor
Department of Computer Science and Engineering
BRAC University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Thesis Co-ordinator
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor and Chairperson
Department of Computer Science and Engineering
Brac University

Abstract

The rapid adoption of MQTT in Internet of Things (IoT) environments has increased the attack surface of resource-constrained systems, making reliable intrusion detection a critical challenge. While prior studies primarily emphasize detection accuracy under balanced or synthetic conditions, fewer address false positive behavior and operational realism in MQTT deployments. This thesis presents a rigorous benchmarking framework for MQTT based intrusion detection using real world traffic with preserved class imbalance. Four machine learning models are evaluated. Logistic Regression, Support Vector Machine, Random Forest, and LightGBM are included. All models are tested under the same conditions. False positive rate is treated as a main metric because it has direct operational cost. The results show clear trade-offs. High recall can be achieved by linear models. However, many false positives are also produced. A false positive rate of about 0.59 is observed. This makes deployment difficult. Ensemble models show different behavior. They cut false alarms substantially and among them LightGBM achieves the lowest false positive rate at 0.2058 while still maintaining competitive detection performance. Transparency is addressed through SHapley Additive exPlanations SHAP applied to the final model. The explanations show that transport layer features such as packet length and inter packet timing dominate model decisions. They also reveal non linear interaction effects which help explain why tree based models outperform linear ones. By combining imbalance aware evaluation transparent hyperparameter reporting and explainability driven analysis this work establishes a reproducible and deployment conscious benchmark for future research on MQTT intrusion detection systems.

Keywords: MQTT, Intrusion Detection System, Internet of Things, Machine Learning Benchmarking, False Positive Rate, Class Imbalance, LightGBM, Explainable AI, SHAP, Cybersecurity

Table of Contents

Declaration	i
Approval	ii
Abstract	iii
Table of Contents	iv
List of Figures	vii
List of Tables	viii
Nomenclature	ix
1 Introduction	1
1.1 Background	1
1.2 Motivation	2
1.2.1 Problem Statement	2
1.3 Objectives	3
1.4 Methodology Overview	3
1.4.1 Stage 1: Data Preparation and Preprocessing	4
1.4.2 Stage 2: Machine Learning Model Evaluation	4
1.4.3 Stage 3: Explainability and Model Interpretation	4
1.5 Evaluation Protocol	4
1.6 Scope and Limitations	5
1.6.1 Scope	5
1.6.2 Limitations and Boundaries	5
2 Literature Review	6
2.1 Background and Fundamental Concepts	6
2.2 Review of Existing Research	8
2.2.1 IoT Intrusion Detection Systems: Overview	8
2.2.2 Intrusion Detection in MQTT-Based IoT Environments	9

2.2.3	Machine Learning Approaches for IoT Intrusion Detection . . .	11
2.2.4	Deep Learning and Transformer-Based IDS: Accuracy Versus Deployability	13
2.2.5	Explainable Artificial Intelligence in IoT Intrusion Detection Systems	13
2.3	Comparative Analysis and Research Gaps	15
2.3.1	Trends in Lightweight and Classical ML-Based IDS	15
2.3.2	Deep Learning and Transformer-Based IDS: Accuracy Versus Deployability	16
2.3.3	Federated Learning-Based IDS as a Distinct Research Paradigm	16
2.3.4	Dataset Realism, Evaluation Bias, and Explainability Gaps . .	17
2.3.5	Identified Research Gaps and Thesis Motivation	17
3	Requirements, Impacts and Constraints	20
3.1	Final Specifications and Requirements	20
3.1.1	Technical Specifications	20
3.1.2	Data Requirements	21
3.1.3	Methodological Requirements	21
3.1.4	Software and Resource Requirements	21
3.2	Societal Impact	22
3.3	Environmental Impact	22
3.4	Ethical Issues	22
3.5	Standards and Best Practices	23
3.6	Risk Management	23
3.7	Economic Analysis	23
4	Design Process and Methodology Overview	24
4.1	Preliminary Design and Model Specification	29
4.1.1	Model Roles and Design Intent	29
4.1.2	Hyperparameter Tuning Strategy	29
4.1.3	Model Configuration Summary	30
4.2	Data Collection and Dataset Description	31
4.2.1	Threat Model and Attack Coverage	32
4.2.2	Benchmarking Implications and Dataset Limitations	32
4.2.3	Data Preprocessing and Feature Engineering	33
4.2.4	Label Encoding and Problem Reformulation	37
4.2.5	Handling of Class Imbalance	38
4.2.6	Feature Cleaning and Transformation	39
4.3	Implementation and Benchmarking Framework	40
4.3.1	Benchmarking Procedure	41

4.3.2	Evaluation Metrics and Measurement Discipline	42
4.3.3	Computational Efficiency and IoT Deployment Feasibility . . .	42
5	Result Analysis	46
5.1	Results and Analysis	46
5.1.1	Performance Evaluation	46
5.1.2	Analysis of Model Behavior and Design Trade-offs	52
5.1.3	Explainability and Feature Contribution Analysis	55
5.1.4	Critical Analysis	59
5.2	Summary of Findings	60
5.2.1	Operational and Methodological Implications	61
5.2.2	Research Contributions	61
6	Conclusion	72
6.1	Summary of Contributions	72
6.2	Boundaries	72
6.3	Future Work	73
	Bibliography	76

List of Figures

2.1	Thematic taxonomy of IoT intrusion detection system literature . . .	8
4.1	Overall experimental methodology pipeline for MQTT-based intrusion detection benchmarking.	25
4.2	Multiclass distribution of attack categories within the MQTTEEB-D dataset.	34
4.3	Categorical composition of the engineered feature set.	36
4.4	Binary class distribution of benign and malicious traffic in the MQTTEEB-D dataset.	38
5.1	Precision–recall comparison of evaluated models on the MQTTEEB-D test set	47
5.2	Comparative analysis of false positive rates across all evaluated models.	48
5.3	Strategic positioning of evaluated models in recall versus false positive rate space.	49
5.4	ROC operating points corresponding to the selected decision thresholds for each model.	50
5.5	Precision versus recall trade-offs across evaluated models.	51
5.6	Evolution of model performance from linear baselines to ensemble-based classifiers, highlighting changes in false positive rate (left) and F1-score (right).	52
5.7	Comparative analysis of Random Forest and LightGBM illustrating false positive rate reduction alongside comparable F1-score performance and training time.	54
5.8	SHAP summary plot for the LightGBM model	56
5.9	Mean absolute SHAP values for top features	56
5.10	SHAP dependence plot for <code>tcp_len</code> colored by <code>tcp_time_delta</code> . . .	57
5.11	Logistic Regression Mean SHAP 	58
5.12	SVM (Balanced) Mean SHAP 	58
5.13	Random Forest Mean SHAP 	59

List of Tables

2.1	Characteristics of IoT environments and security implications	9
2.2	Limitations of signature-based IDS in IoT environments	9
2.3	Limitations of anomaly-based IDS in IoT environments	10
2.4	Emerging IDS research directions for IoT and MQTT	10
2.5	Model family trade-offs in IoT intrusion detection	11
2.6	Comparative synthesis of machine learning-based intrusion detection studies in IoT environments	18
4.1	Methodological design principles and corresponding justifications . . .	28
4.2	Hyperparameter search space for Random Forest classifier	30
4.3	Hyperparameter search space for LightGBM classifier	31
4.4	Dataset overview (MQTTEEB-D)	31
4.5	Collection environment characteristics	32
4.6	Dataset variant selection used in this study	32
4.7	Class distribution in the MQTTEEB-D dataset	32
4.8	Attack behaviors captured in the dataset	33
4.9	Dataset coverage characteristics	33
4.10	Classification formulation used in this study	34
4.11	Feature preprocessing steps	35
4.12	Label transformation scheme	35
4.13	Feature group composition	36
4.14	Feature group and attack evidence	36
4.15	Explainability findings	37
4.16	Binary label encoding scheme for MQTT intrusion detection	37
4.17	Problem reformulation rationale and operational impact	38
4.18	Class distribution after preprocessing	38
4.19	Class imbalance handling strategy	39
4.20	Implications of preserving class imbalance	39
4.21	Feature categorization in the MQTTEEB-D dataset	39
4.22	Benchmarking design philosophy	40
4.23	Data splitting and reproducibility controls	40

4.24	Test-set isolation rules	41
4.25	Evaluation metrics and operational relevance	42
4.26	Metric computation settings	43
4.27	Reproducibility and reporting controls	43
4.28	Training-time characteristics of evaluated models (offline)	44
4.29	Inference-time behavior by model type	44
4.30	Relative computational characteristics of evaluated models	44
4.31	Memory and scalability properties	45
4.32	Deployment-oriented trade-off between detection power and computational cost	45
5.1	Performance comparison of evaluated models on the MQTTEEB-D test set (training time is measured in seconds; for Random Forest and LightGBM it reflects the total hyperparameter search runtime in the described tuning setup)	46
5.2	Comparative analysis of research elements (Part 1).	62
5.3	Comparative analysis of research elements and reporting standards (Part 2).	63
5.4	Positioning of this thesis relative to representative MQTT and IoT intrusion detection studies (Part 1 of 2).	65
5.5	Positioning of this thesis relative to representative MQTT and IoT intrusion detection studies (Part 2 of 2).	66
5.6	System-level comparison between baseline linear model and proposed LightGBM-based framework	67
5.7	Numerical Comparison of IDS Studies	68
5.8	Computational Requirements and Lightweight Justification	68

Chapter 1

Introduction

1.1 Background

Internet of Things (IoT) has become an instrumental paradigm of contemporary cyber-physical systems, allowing them to interconnect on a large scale, by connecting heterogeneous devices and urban, industrial, healthcare, and environmental monitoring domains. These systems are based on the sustained information communication among resource-constrained devices, gateways, and cloud services to enable real-time monitoring and automated decision-making.

As a solution to the limitations of IoT settings, lightweight communication protocols have been created, one of which is Message Queuing Telemetry Transport (MQTT) that has become widely used because of its publish-subscribe architecture and minimal communication overheads. Due to these properties, MQTT is widely applied in real-world IoT systems, especially within resource-limited and latency-sensitive systems [7]. Although MQTT has the advantage of efficiency, it was not initially created with the goal of enforcing the comprehensive security as a major priority. Rather, the protocol uses external protocols like Transport Layer Security (TLS) and access control to the broker sides. Misconfigurations, inadequate authentication mechanisms, and inadequate access control policies are also highly common in real-world deployments and expose MQTT-based IoT systems to protocol-specific attacks [19]. Intrusion detection systems (IDS) thus are very essential in the architecture of IoT security. In contrast to preventive security measures, IDS are aimed at tracing the behavior of traffic and detecting anomalies that results in attacks. Nevertheless, the classic signature-based and rule-based IDS models find it difficult to match the dynamics of the traffic, the heterogeneity of devices and the emerging threats nature of IoT networks. This has inspired the growing popularity of machine learning-based intrusion detection methods specifically designed to address MQTT-based IoT settings [7].

1.2 Motivation

This study is motivated by the fact that there is an increased operational risk involved with MQTT IoT implementations. Both empirical studies and testbed experiments have demonstrated that MQTT infrastructures are oftentimes targeted by attacks such as denial-of-service, brute-force authentication, malformed data injection, publish flooding, and packet rejection, which usually lead to the disruption of service, as opposed to outright system compromise [19].

Despite the fact that machine learning-driven intrusion detection systems have proved to have good detection ability, their application to IoT settings remains challenging. Much of the available literature is based on simulated or partially synthetic datasets which do not reflect the variability, noise, and temporal dynamics contained in actual MQTT traffic [7], [19]. Moreover, many studies place emphasis on detection accuracy as the most important measure of evaluation, and ignore operationally critical factors such as false positive behavior and analyst interpretability. Security-critical IoT environments require that analysts have the capability to comprehend why a given traffic instance is considered malicious in order to authenticate alerts, conduct forensic analysis, and refine security policies. Previous studies have indicated that many machine learning-based IDS operate as black boxes, which hinders transparency and reduces analyst trust. The combination of explainable artificial intelligence methods has hence been cited as a major requirement in enhancing the interpretability and usability of MQTT-oriented intrusion detection systems [25].

1.2.1 Problem Statement

Even with the large body of work on intrusion detection in IoT networks several important gaps remain.

One recurring issue is the heavy reliance on synthetic or protocol agnostic datasets. Many studies evaluate IDS performance on traffic which yet does not reflect the complexity or evolving nature of real world MQTT communication and attack behavior.[7]. Another limitation is the absence of standardized benchmarking studies that compare multiple machine learning models under consistent evaluation protocols in MQTT specific settings [11].

There is also the problem of opacity. A large portion of existing intrusion detection methods operate as black box models and provide little insight into which features or traffic patterns actually drive their decisions [22]. That lack of interpretability restricts practical adoption especially in security critical IoT environments where analysts need explanations for forensic investigation and trust building. On top of this recently released real world MQTT datasets still lack comprehensive baseline

evaluations which leaves researchers without clear reference points for comparison [19].

So the core issue becomes clear. What is missing is a systematic and explainable evaluation of machine learning based intrusion detection models on real MQTT based IoT environments. Closing this gap is essential if IDS solutions are to move from experimental setups toward reliable and deployable systems.

1.3 Objectives

The objective of this research is to evaluate the performance of selected machine learning models for intrusion detection in MQTT-based IoT environments using a real-world dataset. The study focuses on comparative performance analysis, false positive behavior, and basic model interpretability to assess practical applicability. To be specific, the objectives of this study are:

- To evaluate the performance of selected machine learning models for detecting intrusions in MQTT-based IoT traffic.
- To compare the models using standard intrusion detection metrics, with particular emphasis on false positive rate.
- To analyze performance trade-offs among the evaluated models under realistic class imbalance conditions.
- To interpret the behavior of the best-performing model using explainable AI techniques.

1.4 Methodology Overview

The proposed research will employ the use of a data-driven experimental approach to measure and interpret machine learning-based intrusion detection systems in MQTT-based IoT environments. A unified learning system is used in order to guarantee uniformity in training, evaluation, and preprocessing of all models. The overall methodology is organized into sequential phases and among them is data preparation, model evaluation, explainability analysis, and performance analysis.

A real-world MQTT intrusion detection dataset is used to ensure realism and practical relevance. Various classical machine learning models are tested and trained under the same conditions in order to make fair comparison. Explainable artificial intelligence techniques are used on analyzing model behavior and extracting understanding of feature contributions to intrusion detection decisions.

1.4.1 Stage 1: Data Preparation and Preprocessing

The former stage deals with the process of dataset preparation to train and evaluate the model. This entails cleaning up of data, handling missing values, normalization of numeric features and coding of categorical data. Preprocessing pipeline is implemented to generate similar results in all experiments to achieve a system of uniformity and reproducibility. Label encoding is carried out to derive the intrusion detection task as a supervised classification problem.

1.4.2 Stage 2: Machine Learning Model Evaluation

During the second step, several classical machine learning models are fitted and tested for intrusion detection. The effectiveness of these models is the basis of their selection based on compositional efficiency, resource efficiency, and appropriateness to tabular analysis in resource-constrained IoT conditions. All the models are trained with the same dataset partitioning plan and preprocess channel to make a fair comparison. Model performance is measured against the standard intrusion detection metrics.

1.4.3 Stage 3: Explainability and Model Interpretation

The third phase involves the explainable artificial intelligence-based techniques to interpret the predictions of the machine learning models. SHapley Additive exPlanations (SHAP) are utilized to analyze features at global and instance-level importance. This step is to determine important MQTT traffic features affecting detection results and to compare the dependence of various models on individual features in the detection of intrusions.

1.5 Evaluation Protocol

The data partitioning and the experimental setup are stipulated by the evaluation protocol, quality measures, and reproducibility. A stratified train test split is employed to maintain class distribution. Measures are used in determining model performance as accuracy, precision, recall, F1-score and false positive rate. All experiments are done using fixed random seeds to achieve reproducibility.

1.6 Scope and Limitations

1.6.1 Scope

The study is on intrusion detection in MQTT-based IoT environments with the use of monitored machine learning model. The research measures classical machine learning methods of separating benign and malicious MQTT traffic with the help of a real-world data that were obtained in a live IoT testbed. Experiments are conducted in a centralized, offline setting to facilitate controlled and reproducible comparison at the network-flow and protocol-feature level, with emphasis on MQTT communication behavior.

1.6.2 Limitations and Boundaries

The design of this thesis is based on providing a reproducible, deployment conscious benchmark on MQTT intrusion detection. The conditions are therefore stipulated by the following boundaries at which the given results are to be construed:

- **Basis of Data:** Data is appraised based on MQTTEEB-D real-world MQTT data gathered on a certain deployment of an IoT testbed.
- **Protocol coverage:** The protocol analysis is MQTT-based; cross-protocol generalization (e.g., CoAP/AMQP) is not researched.
- **Offline benchmarking:** In experiments, a fixed held-out test set is used to enable controlled comparison; online adaptation and real-time latency are not within the current scope.
- **Explainability interpretation:** SHAP offers feature-attribution evidence. It is not a method of causal explanation of the learned model.

Nevertheless, the boundaries notwithstanding, the study has a solid and practical foundation integrating field MQTT traffic, a combined evaluation protocol, and deployment relevant measures (such as false positive behavior). The framework is readily extensible due to the availability of more datasets and deployment cases.

Chapter 2

Literature Review

2.1 Background and Fundamental Concepts

To set the stage for prior work on intrusion detection in IoT environments this section outlines the core concepts that underpin the present study. The focus is on four closely related areas. The characteristics of IoT networks. The 'operating principles' and security implications of the MQTT communication protocol. Basic intrusion detection approaches. And the role of machine learning within intrusion detection systems. Together these topics provide the technical grounding for the literature review that follows.

Internet of Things (IoT) Networks

The Internet of Things is commonly described as a paradigm in which physical devices sensors and actuators are interconnected through communication networks to enable continuous data collection monitoring and automated control. It now underpins a wide range of application domains including smart homes healthcare monitoring industrial automation smart grids and intelligent transportation systems. In practice IoT environments look very different from traditional computing infrastructures. Massive device diversity, constant data exchange, and strong resource limits are involved. These characteristics lead to several effects. Many IoT devices are built with limited processing power, memory, and energy. Because of this, heavy security mechanisms are difficult to deploy. At the same time, many systems are wireless, distributed, and placed in partly unprotected locations. As a result, exposure to network-based threats is increased. Denial-of-service attacks, unauthorized access, data manipulation, and protocol-level misuse are more likely to occur. For these reasons, intrusion detection and continuous monitoring are not treated as optional features in IoT deployments. They are treated as core parts of keeping system integrity and availability.

MQTT Communication Protocol

Message Queuing Telemetry Transport MQTT is a lightweight messaging protocol built for resource constrained devices and low -bandwidth networks. It uses a publish subscribe communication model. Clients do not talk to each other directly. Everything flows through a centralized broker. Publishers send messages to specific topics. Subscribers receive messages by subscribing to topics they care about. The result is decoupled and scalable communication.

MQTT defines a small set of control packets such as **CONNECT** **PUBLISH** **SUBSCRIBE** and **DISCONNECT** that handle session setup and message transfer. The protocol also supports multiple Quality of Service QoS levels so reliability can be traded against communication overhead.

The design is intentionally simple and efficient. That is why MQTT fits IoT so well. The flip side is that built in security is minimal. That simplicity leaves room for protocol level abuse which is exactly why dedicated security monitoring becomes necessary in MQTT based environments.

Intrusion Detection Systems

At a basic level, an intrusion detection system is used to monitor activity. Malicious behavior and policy violations are searched for. IDS are commonly grouped by detection method. In signature-based IDS, observed behavior is compared with known attack patterns. Low false positive rates are usually produced. However, new or changing attacks are often missed. In anomaly-based IDS, a different approach is used. Normal behavior is first learned. Any strong deviation is then flagged as a possible intrusion. Because of this, unknown attacks can be detected more easily. The trade-off is that more false alarms are often produced. IDS are also grouped by where they are deployed. In network-based IDS, traffic moving through the network is analyzed. In host-based IDS, activity on individual devices is monitored. In IoT environments, network-based IDS are often preferred. Heavy computation is not pushed onto small devices. A central view of network activity is also provided.

Machine Learning in Intrusion Detection

Clear limits are present in rule-based and signature-based methods. To overcome these limits, machine learning is used in intrusion detection systems. Patterns are learned from data. This allows attacks to be detected even when traffic becomes complex or when attack methods change.

Supervised learning are methods depend on labeled datasets to separate normal traffic from malicious activity. Unsupervised and semi supervised methods focus on

anomaly detection and do not require explicit attack labels.

From a formulation standpoint intrusion detection is usually framed as either binary or multi class classification. Binary classification separates normal from malicious traffic. Multi class classification attempts to identify specific attack categories. This choice affects system complexity interpretability and deployment feasibility. So it matters when interpreting how machine learning has been applied in prior IoT and MQTT focused intrusion detection research.

2.2 Review of Existing Research

2.2.1 IoT Intrusion Detection Systems: Overview

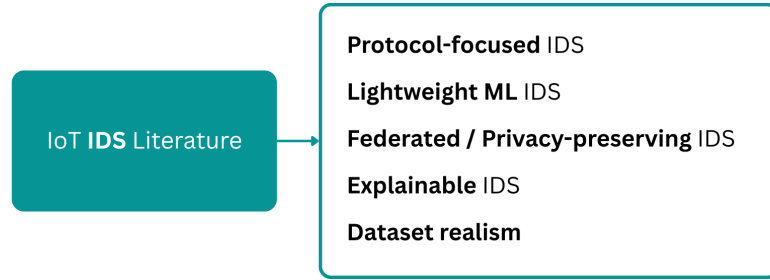


Figure 2.1: Thematic taxonomy of IoT intrusion detection system literature

To clarify intrusion detection challenges in IoT systems and to place this study within existing research, key features, limits, and method trends are summarized in tables. In this way, the problem and motivation are shown in a clear and structured form.

The main features of IoT environments are shown in Table 2.1. The limits of signature-based IDS and anomaly-based IDS are summarized in Table 2.2 and Table 2.3. When these tables are viewed together, the need for other IDS approaches is made clear. The related research directions and model trade-offs are shown in Table 2.4 and Table 2.5.

In Figure 2.1, one main pattern is shown clearly. Much recent work is being directed toward protocol-focused intrusion detection. This is seen especially for lightweight IoT messaging protocols such as MQTT. The main idea is simple. The meaning and structure of a protocol affect how attacks are performed. If only general traffic

Table 2.1: Characteristics of IoT environments and security implications

Aspect	Observation
Device types	Heterogeneous
Resources	Limited (compute, memory, energy)
Communication style	Continuous and broker-mediated
Protocols	Lightweight and always-on (e.g., MQTT)
Security model	Perimeter-based assumptions often fail
Main defense	Continuous monitoring via IDS

Table 2.2: Limitations of signature-based IDS in IoT environments

Property	Outcome
Detection method	Pattern matching against known signatures
False positives	Typically low
Zero-day attacks	Poor handling (unknown patterns not recognized)
Signature updates	Costly and operationally difficult at scale
Standalone suitability	Not sufficient for dynamic IoT threats

statistics are examined, part of the attack behavior is missed. Because of this, purely generic feature sets are no longer seen as sufficient.

In Figure 2.1, growing interest in federated and privacy-preserving IDS is also shown. These approaches mainly change how models are trained. They do not change how intrusions are defined or detected. For this reason, they are treated as secondary in this study. They are used for background context but are not the main focus.

Overall, Figure 2.1 is used as a guide for organizing the literature that follows. Studies are not discussed alone. They are placed within larger research trends.

2.2.2 Intrusion Detection in MQTT-Based IoT Environments

A new technology named Message Queuing Telemetry Transport (MQTT) has proven to be one of the most widely used communication protocols in IoT applications because of its lightweight publish-subscribe design, low bandwidth needs, and applicability to resource-constrained devices. MQTT has widespread application in applications like smart homes, industrial monitoring and sensor networks. However, the protocol applies the efficiency of the communications over the security, thus MQTT-based systems are especially prone to attacks depending on protocols.

One of the main discoveries throughout the literature is that the conventional network intrusion detecting technologies are not well-adapted to MQTT traffic. As compared to traditional client-server protocols, MQTT is based upon persistent connections, communication mediated by a broker, and subject-based message broadcasting. MQTT infrastructure attacks are thus expressed by abnormal publish rates, overconnection efforts, poisoned control packets, or illegal access of a topic. These

Table 2.3: Limitations of anomaly-based IDS in IoT environments

Property	Outcome
Detection method	Deviation from learned normal behavior
Unknown attacks	Potentially detected
False positives	Often high under heterogeneous device behavior
Sensitivity to traffic variation	High (legitimate behavior varies widely)
Design complexity	High (requires careful modeling and evaluation)

Table 2.4: Emerging IDS research directions for IoT and MQTT

Direction	Motivation
Protocol-aware detection	Capture protocol semantics and attack mechanisms
Lightweight models	Fit resource limits and enable near-real-time operation
Explainable IDS	Improve analyst trust and forensic usefulness
Realistic datasets	Improve external validity and reduce evaluation bias

behaviors are often subtle, protocol dependent, and in this case, this necessitates intrusion detection mechanisms that explicitly represent MQTT semantics as opposed to generic traffic statistics.

The initial MQTT-driven intrusion detection studies were mainly based on rule-based or monitoring techniques which are threshold-driven. Such strategies proved that simple indicators like the rate of publication or connection might be useful in detecting flooding and denial-of-service attacks. Such methods are computationally efficient but are generally less efficient than theoretical methods, are not very flexible and are very sensitive with regard to configuration, resistance to changing attack methods and various deployment conditions. Consequently, rule-based methodologies have been found not to be adequate to exhaustive MQTT security monitoring. In more recent works, the use of machine learning methods to detect malicious MQTT traffic has become more popular. A protocol-aware supervised learning model features, including distributions of message types, statistics on session duration, and topic usage patterns, have reported better detection performance, relative to rule-based systems. These findings suggest that MQTT-specific families of features should be included into the feature representation greatly boosts the intrusion detection capability.

Irrespective of these developments there are a number of limitations in the literature. A substantial percentage of MQTT intrusion detection research compares the application of the proposed methods with traffic simulation in controlled conditions. While such datasets facilitate experimentation, they tend to miss the variability, background noise, and mixed benign and malicious behavior in the real-world MQTT deployments. This will lead to an overstatement of performance measures reported effectiveness.

Table 2.5: Model family trade-offs in IoT intrusion detection

Model Type	Detection Power	Cost	Interpretability
Deep learning	High	High	Low
Classical ML	Moderate-High	Low	High

The other common problem is that MQTT intrusion detection has been formulated as a multi-class problem of classifying classes where the individual attack types are considered as independent classes. In spite of being analytically informative, this formulation is not always in agreement where the main goal is to quickly distinguish harmless and dangerous action. Binary classification has more to offer deployment-focused abstraction, in which timely alerting and simplified responding are facilitated workflows, but there is limited research on MQTT-oriented studies.

Explainability is also largely in the absence of MQTT intrusion detection studies. Although some works have high accuracy of detection, not many are insightful whose protocol features or traffic behaviors are used to make model decisions. This lack reduces trust in the analysts and impedes the process of forensic investigation, especially in security critical conditions where it is important to know the purpose of alerts.

Though, new research has presented the real-world MQTT data and implemented explainable learning, classical machine learn benchmarking. The use of models under a common assessment framework is still little. In particular, comparative studies that also take into account detection performance, false positive behavior, and explainability in real-world MQTT environments are scarce. Comparative experimental tests of IoT messaging protocols reveal that MQTT is a lightweight protocol, and sticky session behavior make it stand out compared to other similar protocols like CoAP, resulting in protocol-specific security implications [1].

To conclude, the literature available proves that machine learning-based MQTT network-based intrusion detection through learning is feasible and exposes critical gaps, connections to dataset realism, task formulation and interpretability. These gaps motivate the current study is concerned with binary intrusion detection via real-world MQTT traffic, and systematic analysis of classical machine learning models and verbal exposition analysis.

2.2.3 Machine Learning Approaches for IoT Intrusion Detection

Machine learning has become the dominant approach in IoT intrusion detection largely because it can cope with messy traffic patterns and evolving attack behavior. Unlike signature based systems ML driven IDS learn discriminative structure

directly from data. That means they are not limited to recognizing only what has already been seen. They can also present previously unseen attacks.

Looking across the literature most works fall into three broad groups. i. Classical machine learning models ii. ensemble based approaches and iii. deep learning architectures. Each comes with its own trade offs between detection performance, computational cost and deployment feasibility.

Early ML based IDS for IoT settings relied heavily on supervised models such as decision trees, random forests and gradient boosting. A representative example, work of Jin et al. who proposed *SwiftIDS*. Their system uses LightGBM with a parallel detection mechanism to support real time operation in high throughput environments. The key takeaway is practical, such Gradient boosting models can reach competitive detection accuracy. They keep preprocessing and inference latency lower than heavier frameworks. In other words, classical ensemble based ML can deliver strong performance without providing heavy operational overhead. That makes this family of models a good choice for resource constrained IoT deployments [6].

To further reduce computational cost, more recent work gave importance to lightweight and flow-based learning paradigms. Mutambik introduced a flow-based anomaly detection framework that models benign traffic behavior rather than depending on large labeled attack datasets. But tested results report effective anomaly detection. Also low false positive rates. These findings mainly shows the feasibility of IDS designs, giving importance to efficiency and scalability over small accuracy gains. In this regard, the study shows the growing focus on deployable IDS solutions. This aligns with real-world IoT difficulties. [14].

In contrast, quite a few body of literature has examined deep learning (DL) architectures to improve detection accuracy using learning temporal and spatial patterns in network traffic. Vijayan and Sundar proposed an IDS for MQTT-based IoT environments. The system uses autoencoders and LSTM-based deep belief networks optimized through heuristic algorithms. It achieves high detection accuracy on MQTT datasets. This result shows that deep learning can support protocol-aware intrusion detection. However, the architecture requires multi-stage training and high computational cost. As a result, scalability and real-time deployment in complex IoT environments remain not-explored. This paper is cited to demonstrate the accuracy–complexity trade-off in deep learning–based IDS [8].

2.2.4 Deep Learning and Transformer-Based IDS: Accuracy Versus Deployability

Using transformer architectures, recent paper have extended IDS design beyond earlier deep learning approaches. These models capture long-range dependencies in network traffic. They also achieve improved anomaly detection on benchmark datasets. However, actual evidence indicates greater memory usage.. Inference latency is higher. Training overhead is substantial. Such costs are especially restrictive for resource-limited IoT devices. Consequently, despite strong detection performance, transformer-based IDS face practical barriers to lightweight deployment [17].

Along these developments there some studies/papers have begun integrating federated learning and privacy preserving training into IoT intrusion detection frameworks. These approaches tackle important concerns around data privacy and decentralized learning. But they also bring their own complications. Communication overhead increases. Convergence can become unstable. Non IID data distributions across devices add another layer of difficulty.

The key point is that federated learning changes how models are trained rather than how intrusions are detected. For that reason these works are considered contextual background in this study rather than a central focus and are not explored in depth here.

Overall, the literature shows that although deep learning and transformer-based models achieve high detection accuracy, their computational demands and system complexity limit practical use in many IoT settings. By contrast, classical and ensemble-based machine learning models offer a more favorable balance among performance, efficiency, and interpretability. These observations motivate the present study's focus on evaluating lightweight machine learning classifiers under realistic MQTT traffic conditions, with explicit attention to operational feasibility and subsequent explainability analysis.

2.2.5 Explainable Artificial Intelligence in IoT Intrusion Detection Systems

With the increased integration of machine learning models into intrusion detection systems (IDS), issues of transparency, trust and forensic usability have gained prominence. High detection accuracy alone is insufficient for security-critical applications, in which analysts are required to know the reasons a certain traffic instance is classified as malicious. This has inspired the adoption of Explainable Artificial Intelligence (XAI) strategies into IDS frameworks such that it allows post-hoc interpretation of

model decisions and informed security response support.

The key body of literature in XAI is that black-box intrusion detection models, ensemble and deep learning methods in particular, are commonly very successful in predictive precision in exchange for readability. Ahmed et al. have shown that hybrid bagging and boosting models boosted with SHAP-based feature importance analysis are able to maintain high detection accuracy without giving meaningless explanations of model behavior. They conclude that SHAP is able to detect influential traffic features without impaired detection performance. This work is cited to determine that explainability is capable of being integrated into ensemble-based IDS without sacrificing predictive strength, especially applicable to operational deployment [9].

Comparative research has also tested the appropriateness of various XAI techniques in security analysis. Hermosilla and others have made a systematic comparison of SHAP and LIME to analyze forensic intrusions. SHAP is shown to generate more reliable and consistent explanations between samples, but LIME displays local sensitivity and sampling variability. Their results point to the fact that explanation reliability is one of the critical elements in security contexts where inconsistent interpretations can be deceptive to analysts. This paper is referenced to give validity to the choice of SHAP as a major explainability method in intrusion detection applications [22].

Explainability within certain model families has also been studied in recent research commonly used in IDS. A frame of XGBoost with XAI enhancements was suggested by Dakhil and Cakmak for intrusion detection in IoMT settings. Their framework can be used together with SHAP with high detectability and inherent interpretable feature attributions. According to their findings, classical machine learning models have an advantageous accuracy-to-explainability ratio compared to deep learning approaches. This is one of the works added in favor of the appropriateness of explainable intrusion detection based on tree-based, lightweight classifiers for resource-constrained environments [20].

There has also been a study into explainability in protocol-conscious IDS designs. Zeghida et al. proposed an explainable machine learning-based IDS, XMID-MQTT. In the case of MQTT traffic, protocol-specific features were interpreted using SHAP, such as message types, session duration, and publish-subscribe behavior. Their analysis proved that explanation in terms of protocols aids analyst comprehension of attack mechanisms in MQTT networks. This is used to define the importance and significance of explainability as a direct part of MQTT-centered intrusion detection systems [25].

Regardless of these developments, there are weaknesses indicated in the literature. Many explainable IDS studies stress visualization of feature importance without sys-

tematic evaluation of explainability on operational decision-making or false positive reduction. Moreover, explainability is usually considered as a secondary part instead of an intrinsic component of model assessment, and there is little discussion of how protocol semantics or real-world attack behavior is consistent with explanations. Summarizing the presented research, it is possible to state that XAI methods (especially SHAP) can greatly increase transparency and forensic usefulness of machine learning-based intrusion detection systems. Nevertheless, a shortage of systematic explanation assessment in lightweight, protocol-conscious IDS on real-world MQTT traffic is still present. These observations provide the motivation to the focus of the present study on explaining classical machine learning models with SHAP-based explainability, facilitating effective intrusion detection as well as interpretable decision support in realistic IoT environments.

2.3 Comparative Analysis and Research Gaps

In this section, we synthesize the findings from the literature reviewed in *Section 2.2* to identify dominant research patterns, recurring limitations, and open challenges in IoT intrusion detection. Rather than reiterating individual study results, the analysis compares methodological decisions, evaluation practices, and deployment assumptions across the literature to clearly motivate the research direction pursued in this thesis.

2.3.1 Trends in Lightweight and Classical ML-Based IDS

One of the aspects of current IoT intrusion detection research that is evident is the resurgence of focus on models of lightweight and classical machine learning. Studies focusing on decision trees, random forests, gradient boosting and one-class classifiers regularly show that competitive detection performance can be realized with significantly reduced computational costs as compared to deep learning models. Alomari et al. demonstrated that lightweight ML-based IDS is capable of sustaining high detection accuracy while being practicable to run on small IoT devices, expressing the need to emphasize model simplicity in real-world scenarios [10]. Evidently, Golestani and Makaroff have shown that unsupervised one-class classifiers can provide efficient anomaly detection and minimal training is required, especially when labeled attack data have low availability [13]. “These trends are summarized in Table 2.6.”

This pattern also lines up with what deployment oriented studies report. Stolz et al. examined lightweight IDS deployment on resource constrained hardware and showed that model complexity directly affects latency memory usage and energy consumption [15]. A similar idea is presented by Gadewar et al. IDS effectiveness is judged

based on deployment conditions, not only on accuracy [12]. When these works are looked at together, one conclusion is reached. Realistic IoT intrusion detection is achieved when detection performance and practical limits are both considered.

2.3.2 Deep Learning and Transformer-Based IDS: Accuracy Versus Deployability

Many studies use deep learning for intrusion detection. Neural networks are used because complex patterns can be learned from data. Time-based patterns and feature relationships can be captured. New transformer-based models such as TFKAN have also been proposed. Long-range relationships between features can be modeled by these models [21]. High accuracy is often reported on benchmark datasets for these approaches.

The trade off is cost. Higher accuracy typically comes with increased training complexity greater memory consumption and longer inference time. Many comparative studies still evaluate these models under idealized experimental settings without fully accounting for deployment constraints in real IoT environments. Transformer based IDS offer substantial representational power. But their applicability on resource constrained devices remains limited.

So these studies are useful for defining upper bounds on detection performance. At the same time they reinforce why lightweight and efficient alternatives remain necessary when the goal is practical deployment rather than headline accuracy.

2.3.3 Federated Learning-Based IDS as a Distinct Research Paradigm

Federated learning (FL) as a privacy-maintaining strategy to IoT intrusion detection has increasingly become the focus of recent literature. Khraisat et al. and Anwar et al. showed that federated training has the capacity to alleviate concerns about data-sharing by facilitating cooperative model learning without data collection in a central place [18], [24]. More optimizations are suggested to enhance the stability of convergence under non-IID data distributions [16], [23].

In spite of these benefits, federated learning comes with other problems, including communication overhead, complexity of synchronization, and system-level coordination. In addition, FL is mostly concerned with training architecture rather than detection methodology. These aspects position federated learning-based IDS as a parallel research direction as opposed to a direct alternative to centralized lightweight IDS. Based on this, such approaches are mentioned to frame the larger research landscape but are not followed up in the scope of this thesis.

2.3.4 Dataset Realism, Evaluation Bias, and Explainability Gaps

Across many method types, two problems are seen again and again. One problem is dataset realism. The other problem is evaluation bias. Many survey studies show that a large number of intrusion detection experiments are done using artificial data or very old datasets. Because of this, results often look good in papers. However, similar performance is not seen when systems are used in real environments [11].

At the protocol level, another concern is raised. MQTT-specific behavior is often not well represented in common benchmark datasets. When the protocol is not modeled in a realistic way, reported results become difficult to trust [2], [5].

Explainability is also a weak area. Most intrusion detection research still focuses mainly on detection accuracy. Much less attention is given to explaining why a model makes a certain decision. Because of this, security analysts are left with little help when trying to understand alerts. This reduces forensic value and lowers trust in automated systems, especially in critical IoT settings.

Large survey studies on IoT security architecture help explain why these problems remain. Many IoT systems are built with different protocols and limited native security. These design choices make intrusion detection harder than in traditional networks [3], [4].

2.3.5 Identified Research Gaps and Thesis Motivation

We found some gaps in the following research areas:

- Lightweight machine learning models are so much discussed and almost not tested in a systematic way using realistic MQTT traffic.
- Many studies still circle around accuracy numbers than giving more less attention to false positives deployment cost and day to day feasibility.
- Explainability is often treated as an optional plus point not a core part of protocol-sensitive IoT intrusion detection.
- Truly real world MQTT datasets that mirror actual traffic behavior and attack dynamics but used so much less than they should be.

This is where the motivation for the present paper comes from. The target is not to introduce another yet another complex detection model. *Instead the importance is on a centralized lightweight machine learning based framework can be evaluated on real world MQTT traffic. By adding up SHAP based explainability together plus deployment oriented evaluation metrics, the paper aims to move IDS research a step*

closer, this is actually needed in practical IoT security environments

Reference	Year	Methodology	Dataset	Type	Key Findings	Identified Limitations
Jin et al. [6]	2020	LightGBM with parallel IDS	Benchmark datasets		High detection accuracy with low latency	Evaluated on non-IoT datasets; lacks protocol awareness
Mutambik [14]	2024	Flow-based anomaly detection	Simulated traffic	IoT	Low false positives and efficient detection	Limited attack diversity; synthetic traffic
Alomari et al. [10]	2024	Lightweight ML classifiers	Simulated environment	IoT	Feasible deployment on constrained devices	Absence of real-world validation
Vijayan and Sundar [8]	2023	AE + LSTM-based DBN	Simulated MQTT traffic		High detection accuracy for MQTT attacks	High computational complexity; limited scalability
Almadhor et al. [17]	2025	Transformer-based IDS	Benchmark datasets		Improved anomaly detection accuracy	High memory and inference overhead
Hindy et al. [7]	2021	ML-based MQTT IDS	Controlled experimental setup		Protocol-aware feature effectiveness	Lack of real-world traffic diversity
Zeghida et al. [25]	2025	Explainable ML (SHAP)	MQTT-focused dataset		Improved interpretability of IDS decisions	Limited evaluation of operational impact
Aqachtoul et al. [19]	2025	Real-world MQTT dataset	Real IoT deployment		Realistic traffic and attack representation	Dataset-focused; no IDS benchmarking

Table 2.6: Comparative synthesis of machine learning-based intrusion detection studies in IoT environments

Table 2.6 is presented for more than list prior studies. It shows how choices of methodology dataset selection and evaluation style quietly presented the performance numbers which get reported. When the studies are viewed side by side recurring trends and limitations we see more visible pattern.

One pattern shows up almost immediately. High detection accuracy is most often reported in work that relies on benchmark or simulated datasets. Jin et al. and Almadhor et al. can be examples, as reported strong results using ensemble and transformer based models. But both evaluations are held out on non IoT or generic benchmark data. That data does not help us for real MQTT protocol behavior. Also the diversity of traffic in operational IoT systems. Numbers look impressive here so. The question is how much they would benefit in practice.

Deployment oriented designs tell something different. Lightweight & flow based approaches, by Mutambik and Alomari et al. separately try to reduce computational cost and false positives. That goes well with IoT realities. But Table 2.6 shows that, methods are usually evaluated only on controlled or synthetic traffic again. Without validation on operational networks, we cannot trust the effectiveness, another limitations remain.

Deep learning based MQTT IDS are particularly relevant here. Systems proposed by Vijayan and Sundar capture protocol specific "temporal patterns" that often achieve high accuracy. At the same time a similar trade-off is found. When architectures become more complex, training cost, memory usage and runtime overhead increase.

This issue becomes even more pronounced for transformer based IDS where better detection performance comes with heavy inference and memory demands. This just hardens real time and large scale deployment, not feasible.

Explainability is another area where the table is clear. It appears occasionally, but as a central evaluation criterion. Zeghida et al. are one of the few to integrate SHAP into an MQTT focused IDS. Even then Table 2.6 suggests that explainability is treated as a supporting featurer not a primary objective. Very few research shows whether explanations actually help reduce false positives or improve analyst decisions. Which leaves their operational value largely untested.

Dataset realism may be the biggest gap of all. The MQTTEEB-D dataset introduced by Aqachtoul et al. is a meaningful step forward as it contains real world MQTT traffic. Though most IDS models in the literature are still not evaluated on this type of realistic data. This creates a common disconnection between academic benchmarking and deployment needs.

Putting all of this together the gap becomes clear. Lightweight modeling real world MQTT traffic and explainable intrusion detection are usually studied in isolation. Very few works bring them together in a single framework. This thesis presented to be at that intersection by evaluating lightweight explainable machine learning models on a real world MQTT dataset using deployment based performance metrics.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

A set of technical, methodological, and software requirements is necessary to evaluate the proposed MQTT-based intrusion detection system and ensure its reproducibility. The overall goal is to ensure transparency, reproducibility, and deployment feasibility. These requirements are established on a conceptual pipeline that has been experimented with and evaluated.

3.1.1 Technical Specifications

Programming Environment

The programming language used throughout this research work was mainly Python 3.x. From data preprocessing to model training, and later generating analytical reports, Python-based workflows proved to be reliable and easy to work with. To set up baseline ML models, scikit-learn is utilized. Ensemble-based IDS experiments were conducted using LightGBM. For explainability and feature contribution analysis, the SHAP library was used.

Hardware Assumptions

The training and inference environment setup was done using only CPU. Any heavy-weight technology like graphics processing units (GPUs) or tensor processing units (TPUs) was purposefully avoided to keep the IDS deployable for edge IoT devices. To process tabular MQTT traffic, sufficient memory was assumed.

Model Specifications

As baseline classifiers, Logistic Regression and Support Vector Machine (SVM) were used with standard parameters. In contrast, Random Forest and LightGBM were considered as ensemble classifiers and tuned via hyperparameter optimization. To distinguish between benign and malicious MQTT traffic, binary classification was formulated.

3.1.2 Data Requirements

Dataset Characteristics

The dataset reflects contemporary attack behavior through a real-world MQTT traffic dataset. Along with the realism of data, operational conditions needed to be maintained; therefore, class imbalance was intentionally preserved. Feature extraction was performed from transport-layer and MQTT protocol-level attributes.

Data Partitioning

To maintain class distribution consistency, a stratified train–test split is maintained. A fixed random seed is used to ensure reproducibility. A 70–30 partition was kept between training and evaluation data to prevent leakage.

3.1.3 Methodological Requirements

Evaluation Framework

Primary evaluation metrics include Precision, Recall, F1-score, and False Positive Rate. Specifically, False Positive Rate (FPR) was treated as a first-class metric given its operational impact in MQTT-based systems. Identical and unified conditions were preserved across baseline and ensemble models for comparative benchmarking.

Hyperparameter Optimization

Hyperparameter optimization through stratified cross-validation was applied to ensemble-based models to improve performance under consistent conditions. To guarantee reproducibility, the full hyperparameter search space is explicitly documented.

3.1.4 Software and Resource Requirements

Software Dependencies

Data manipulation relied on NumPy and Pandas. For visualization, Matplotlib was used. For post-hoc explainability analysis, the SHAP library was used.

Reproducibility Requirements

The study maintains and documents its predefined experimental configuration, along with its parameters, and includes a deterministic evaluation of the pipeline.

3.2 Societal Impact

This study is intended to support the development of reliable and interpretable intrusion detection systems for MQTT based environments. In practical security deployments two aspects matter most. False positive reduction and transparency. When automated monitoring systems generate excessive false alarms they become difficult to trust and even harder to operate. By emphasizing false positive control this work contributes toward safer and more dependable security operation.

Explainability adds another important layer. When analysts can understand why traffic is flagged they are better equipped to validate alerts investigate incidents and make informed decisions. In that sense this research supports human centered security rather than replacing human judgment with opaque automation.

3.3 Environmental Impact

The proposed framework does not depend on specialized or otherwise energy intensive hardware. It is software driven and designed around lightweight machine learning models. This choice keeps environmental impact limited therefore manageable.

Many deep learning based approaches require GPU acceleration and sustained high power usage. In contrast the models used in this study rely on CPU based inference and do not require continuous retraining. That combination reduces energy consumption and long term computational overhead.

Lightweight deployment also lowers dependence on centralized cloud infrastructure. Together these design choices align the framework with more sustainable computing practices for IoT security deployments.

3.4 Ethical Issues

The established ethical principles related to data usage, transparency, and responsible AI deployment are addressed in this research work. The dataset contains no personally identifiable information; hence, privacy is not compromised. Preserving data imbalance prevented the study from relying on artificially balanced environments. SHAP is implemented to justify model decision behavior. This methodology

aligns the work with professional standards in cybersecurity research.

3.5 Standards and Best Practices

Best practices in network security are addressed through this proposed framework. During feature extraction, it complies with MQTT protocol specifications. Common IDS evaluation includes precision, recall, and false positive analysis, which is carried out here. Similar to cybersecurity research norms, this study also emphasizes reproducibility and transparency. The experiment alignment is methodological rather than regulatory.

3.6 Risk Management

Potential risks were addressed with cautious steps. To prevent model overfitting and dataset bias, the study adhered to cross-validation and preserved natural data imbalance. Generalization risk was avoided through conservative claims and framed as future work. To reduce operational misuse, explainability and human-in-the-loop interpretation were incorporated.

3.7 Economic Analysis

To keep the proposed benchmarking framework cost-effective and accessible, the study uses open-source tools and eliminates licensing costs. Specialized hardware expenses are also avoided via CPU-based execution. Reduced false positive rates can lower downstream operational costs associated with manual alert handling. Once trained, models can be deployed with minimal marginal cost.

Chapter 4

Design Process and Methodology Overview

The primary objective of this study is to establish a systematic, reproducible, and Deployment-aware IDS using strictly machine learning for MQTT-based environment. This study is vastly different from other optimization-driven studies. Where most conventional method is to prioritize architectural novelty or marginal accuracy gains, this proposed methodology explicitly emphasizes and validates analytical clarity, operational realism, and comparative validity. This method of evaluation proves that the lack of real-world datasets and unrealistic benchmarks has made research complicated in this particular field of MQTT-based intrusion analysis.

MQTT is a publish-subscribe communication protocol for resource-constrained IoT environments. The reason for its wide usability is its low latency, minimal computational overhead, and continuous broker-mediated communication.

The practical constraints of MQTT-based systems hinder computationally intensive deep learning architectures to perform seamlessly, as they were initially developed for traditional enterprise networks. Consequently the methodology in this study follows a constraint driven design philosophy. Methods are chosen based on how MQTT works. High scores are not the main goal. A centralized offline evaluation is used. Testing is kept simple and clear. All models are compared fairly, only model behavior is measured. Training system effects are reduced. Federated learning does offer privacy and decentralization advantages. But it also brings communication overhead convergence instability and non IID data complications that can blur interpretation and make clean cross model comparison difficult.

Basically the trade off is intentional. For this stage of analysis clarity and reproducibility matter more than distributed optimization.

As shown in Figure 4.1, this methodology separates data preparation, model evaluation, and explainability analysis. A unified pipeline was maintained for every model analyzed in this study to ensure consistency and reduce procedural bias. This also

guarantees the observed differences to reflect each model’s intrinsic behaviour.

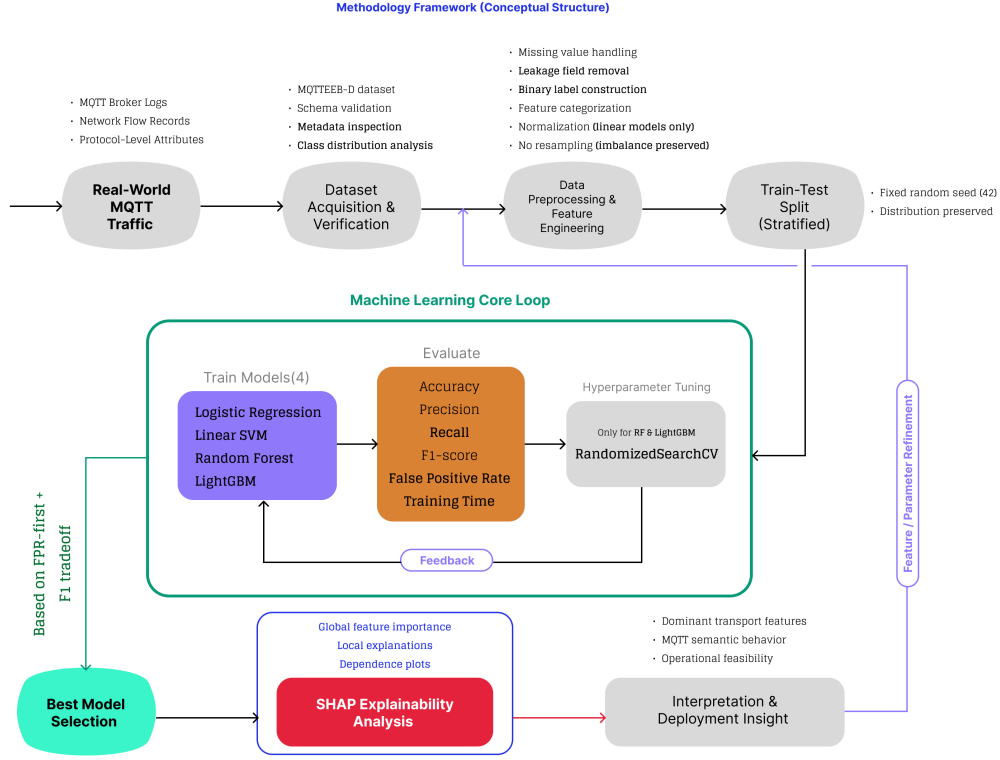


Figure 4.1: Overall experimental methodology pipeline for MQTT-based intrusion detection benchmarking.

End-to-End Methodology (Stage-wise Procedure)

The overall experimental workflow is structured into four sequential stages. The goal of this section is to state precisely what is executed in each stage and what artifacts are produced, so that the pipeline in Figure 4.1 is directly traceable to the reported results in Chapter 6.

Stage 1: Data acquisition and consolidation

This stage establishes a single well-founded dataset to evaluate all experiments. The following steps are performed:

1. **Dataset acquisition:** The MQTTEEB-D dataset was already CSV processed, and initially, the accompanying metadata files were collected from the official release package.
2. **Schema and metadata verification:** To maintain completeness and consistency, the feature columns and label fields are inspected. The provided

metadata, for example, categorical conversion and label-encoding documentation, is used to confirm that the dataset is already numeric and suitable for direct model input.

3. **Consolidation into one benchmark file:** In situations where multiple dataset variants exist, such as loop-level traces versus consolidated files, a single consolidated, a standardized processed representation is designated as the benchmark input for all models to avoid performance drift induced by the variant.
4. **Class distribution reporting:** The class proportions are obtained and then Documented to demonstrate the imbalance and to guide the evaluation process with an emphasis on false positives.

Output of Stage 1: to summarize the initial step, a fixed benchmark dataset (single CSV representation) and documented dataset characteristics were used throughout training and evaluation.

Stage 2: Preprocessing and feature engineering

This stage converts the benchmark dataset into model-ready inputs while preserving protocol semantics and avoiding leakage. The preprocessing steps are intentionally kept lightweight to preserve authenticity.

1. **Leakage-sensitive field removal:** to avoid temporal leakage, timestamp ordering fields are removed.
2. **Binary target construction:** Multi-class labels are reformulated into a binary target (Normal [0] vs. Attack [1]).
(i.e., `Target_binary = 1` when `Target` $\neq$ 3 in the provided label encoding).
3. **Stratified train-test split:** The original class proportions across training and test partitions were to be maintained, as for that stratification was used to split the data
4. **Model-consistent feature scaling:** Specially for distance/linear models (Logistic Regression and Linear SVM) Standardization is applied. On the contrary, Tree-based models (Random Forest and LightGBM) use the same feature values without scaling to avoid wasteful transformations.
5. **Imbalance handling without resampling:** Here, it was very important to preserve class imbalance in data distribution. Hence, class weighting is used instead of oversampling/undersampling with the motive to maintain benchmarking realism.

Output of Stage 2: unified training and test partitions of data. As well as model-ready feature representations like scaled inputs for linear models and unscaled inputs for tree models.

Stage 3: Model training and comparative evaluation

This stage trains all selected models under an identical experimental protocol and calculated the full metric suite used for selection and appraisal:

1. **Baseline model training:** To preserve a reproducible baseline, Logistic Regression and Linear SVM are trained using standard configurations via class weighing.
2. **Selective hyperparameter tuning:** Random Forest and LightGBM are tuned using a fixed RandomizedSearchCV setup and maintain a fixed random seed. Further, to keep the held-out test set fully isolated, tuning is executed exclusively on training folds.
3. **Training-time measurement:** Training time is recorded as an empirical runtime cost (including the hyperparameter search runtime for tuned models) to enable efficiency-aware comparison.
4. **Test-set evaluation:** Each finalized model is evaluated on the same held-out test set. Metrics are computed uniformly on - accuracy, precision, recall, F1-score, and false positive rate , where $FPR = \frac{FP}{FP+TN}$, and probability scores are kept unchanged where available to support ROC/AUC-style analysis. Alongside to tabular metrics, the results are visualized using ROC curves, comparative bar plots, and performance trend charts in the Results chapter.

Output of Stage 3: Outcomes from this stage are selectively trained models, metric tables of each model, and most significantly, comparative visualizations using ROC curves, bar plots, and trend charts. All mentioned in Chapter 6.

Stage 4: Post-hoc explainability analysis

This stage uses a consistent post-hoc framework to explain the decision behavior of the best-performing models. :

1. **Model selection for explanation:** For interpretability analysis, it opts for a best-performing tuned ensemble along with key baselines for contrast.
2. **SHAP-based feature attribution:** SHAP values are computed to measure feature contributions in terms of attack or normal predictions. Summary and comparative plots are generated to identify predominant transport- and protocol-level indicators.

3. **Interpretation and linkage to metrics:** To justify why a model performs well or produces false alarms, explanations are interpreted alongside FPR and F1 behavior. Which supports the final selection argument.

Output of Stage 4: This staged design ensures that detection efficiency , operational limitations , and model interpretability are evaluated concurrently rather than in isolation, to address methodological limitations identified in prior intrusion detection research. Most significantly , the proposed methodology serves as a replicable reference benchmark rather than a deployment-ready intrusion detection solution. We ensured these I. Benchmarking integrity is maintained. II. Experimental settings are kept the same. III. Class imbalance is kept to match real attacks. IV. Only limited model tuning is used. V. Results are realistic and repeatable. VI. Easy interpretation is prioritized. VII. Inflated performance is avoided.

Table 4.1: Methodological design principles and corresponding justifications

Design Aspect	Methodological Choice		Justification
Learning paradigm	Centralized	offline	Enables controlled benchmarking by isolating model behavior from training-architecture effects
Model family	Classical	machine learning models	Balances detection performance, computational efficiency, and interpretability under IoT constraints
Task formulation	Binary classification		Aligns with practical IDS requirements and simplifies false positive rate analysis
Evaluation focus	FPR-sensitive metrics		Reflects operational cost of false alarms in real-world IoT deployments

Table 4.1 summarizes the core methodological principles inspiring the experimental design, precisely correlating each design decision to its underlying rationale. Instead of optimizing for a single performance metric, the framework balances detection capability, deployment feasibility, and interpretability. This structured systematic alignment between deployment and operational constraints in respect of methodological choices establishes the role of the evaluated and recommended framework as a foundational benchmark for MQTT-based intrusion detection research.

4.1 Preliminary Design and Model Specification

The selection and configuration of ML models assessed in this study and discussed in this segment of the study prioritize a methodological justification rather than architectural based on algorithms . This thesis focuses on three criteria for benchmarking: (i) suitability for tabular, protocol-aware network features, (ii) computational feasibility under IoT-oriented constraints, and (iii) interpretability to support operational decision-making and forensic analysis. Unlike intrusion detection studies that prioritize deep learning architectures, the present The work focuses on classical machine learning models to maintain transparency and to keep the experimental pipeline comparable across methods. The MQTT traffic feature has a tabular nature, and this plan of work is more applicable to it. Accordingly, four representative classifiers were selected: Logistic Regression (LR), Linear Support Vector Machine (SVM), Random Forest (RF), and LightGBM. At first, to determine a reference performance under minimal model complexity, Logistic Regression and Linear SVM serve as baseline linear classifiers. On the other hand, Random Forest and LightGBM, which represent tree-based ensembles, have higher expressive capacity. Later the comparative performance report is showed in Chapter 5.

4.1.1 Model Roles and Design Intent

As the first step Logistic Regression and Linear SVM were included as baseline models. And the reason for choosing this model is to represent a linearly separable decision boundary commonly used in resource-constrained intrusion detection systems. These models have negligible computational overhead and have fundamental interpretability, Which makes them suitable for using as reference models to evaluate the marginal benefits of more complex classifiers.

Later, to address the need to capture non-linear feature interactions present in real-world MQTT traffic, Random Forest and LightGBM were selected as ensemble-based models. Notably, LightGBM is more efficient in optimizing gradient boosting implementation, which is generally used for datasets with tabular classification, but still generates very little computational overhead. Along with that, this research incorporates Random Forest and evaluates under identical conditions and feature extraction.

4.1.2 Hyperparameter Tuning Strategy

Random Forest and LightGBM models have higher model capacity and sensitivity to parameter configuration, which is why hyperparameter was applied selectively to these models. Later, standard parameters were used to evaluate Logistic Regression

and Linear SVM to preserve baseline integrity. These design choices are facilitated to prevent excessive optimization of linear models and to enable the ensemble-based methods to demonstrate meaningful outcomes.

To incorporate hyperparameters in Random Forest and LightGBM, Randomized-SearchCV with a fixed random seed is applied in segregated training data. In order to prevent data leakage, the test set was kept isolated throughout the entire experimental process. Most importantly, across all experiments to ensure reproducibility, unified and identical hyperparameter search configuration was applied consistently across all experiments.

Table 4.2: Hyperparameter search space for Random Forest classifier

Hyperparameter			Explored Values
Number of trees (<i>n_estimators</i>)			{100, 200, 300}
Maximum tree depth (<i>max_depth</i>)			{None, 10, 20, 30}
Minimum samples to split (<i>min_samples_split</i>)			{2, 5, 10}
Minimum samples per leaf (<i>min_samples_leaf</i>)			{1, 2, 4}
Feature selection (<i>max_features</i>)			{sqrt, log2}
Class weighting (<i>class_weight</i>)			balanced

Table 4.2 documents the complete hyperparameter search space explored for the Random Forest classifier. The selected parameters control tree complexity, feature subsampling, and class imbalance handling, allowing systematic evaluation of bias-variance trade-offs.

By explicitly reporting the full search space instead of only the final optimized settings here - the study removes guess'work around how models were configured. Nothing is hidden behind a single best number. This makes the results easier to reproduce and avoids ambiguity about what was actually tried.

The hyperparameter settings for the LightGBM classifier are shown in Table 4.3. Learning, tree structure, and sampling options are included. The search space is kept small. This reflects the limited resources of MQTT-based IoT systems. Listing all ranges improves transparency and makes the results easy to reproduce.

4.1.3 Model Configuration Summary

After tuning, the best Random Forest and LightGBM settings were chosen using cross-validated training results. These settings were tested on a held-out test set

Table 4.3: Hyperparameter search space for LightGBM classifier

Hyperparameter	Explored Values
Number of boosting rounds (<i>n_estimators</i>)	{100, 200, 300}
Learning rate (<i>learning_rate</i>)	{0.01, 0.05, 0.1}
Maximum tree depth (<i>max_depth</i>)	{-1, 10, 20, 30}
Number of leaves (<i>num_leaves</i>)	{31, 63, 127}
Minimum child samples (<i>min_child_samples</i>)	{20, 50, 100}
Row subsampling (<i>subsample</i>)	{0.7, 0.8, 0.9}
Column subsampling (<i>colsample_bytree</i>)	{0.7, 0.8, 0.9}
Class weighting (<i>class_weight</i>)	balanced

Table 4.4: Dataset overview (MQTTEEB-D)

Item	Description
Dataset name	MQTTEEB-D
Release year	2025
Domain	MQTT-based IoT intrusion detection
Data source	Real-world testbed
Traffic type	Benign and malicious

with no further changes. Logistic Regression and Linear SVM used their default settings in all experiments. Thus, comparisons are fair and ensures that differences come from the models themselves, not from extra tuning.

4.2 Data Collection and Dataset Description

The main dataset properties are shown in Table 4.4. The data collection setup is presented in Table 4.5. When these tables are viewed together, the realism of the dataset can be better understood. The dataset version selected for this study is shown in Table 4.6, based on these properties.

Table 4.7 shows a strong class imbalance, with attack traffic being the majority. This imbalance is not corrected. The original distribution is kept. Real systems can face long attack periods. Models therefore need to work under this condition. This choice keeps evaluation realistic and shows why accuracy alone can be misleading.

Table 4.5: Collection environment characteristics

Aspect	Description
Environment	Live IoT testbed
Components	MQTT broker, clients, attack agents
Traffic nature	Real network traffic
Temporal dependencies	Preserved
Protocol semantics	Preserved
Benign-malicious overlap	Present
Class imbalance	Present

Table 4.6: Dataset variant selection used in this study

Dataset Variant	Status	Reason
Raw loop-level captures	Not used	High complexity
Consolidated processed dataset	Selected	Unified representation
File used	Selected	MQTTEEB-D_dataset _All_Processed.csv (baseline dataset)
Feature state	—	Cleaned and encoded
Usage	—	All experiments

4.2.1 Threat Model and Attack Coverage

The attack behaviors present in the dataset are shown in Table 4.8. The overall coverage and limitations are presented in Table 4.9. When these tables are viewed together, the practical scope of the dataset can be understood. Based on this scope, the classification formulation used in this study is shown in Table 4.10.

In Figure 4.2, an imbalance between attack types is shown. The MQTTEEB-D dataset is not evenly distributed. Most traffic is from malformed packet attacks. Flooding attacks and slow-rate attacks are seen much less. This imbalance is not by chance. It is similar to real MQTT systems, where malformed messages are found more often than large denial-of-service attacks.

4.2.2 Benchmarking Implications and Dataset Limitations

When only one real-world dataset is used, limits are placed on how far the results can be applied. IoT deployments are different. Broker settings, network layout, and

Table 4.7: Class distribution in the MQTTEEB-D dataset

Class	Proportion (%)
Attack traffic	~92%
Normal traffic	~8%

Table 4.8: Attack behaviors captured in the dataset

Attack Aspect	Description
Target	MQTT infrastructure
Connection behavior	Abnormal connection attempts
Message behavior	Excessive publish rates
Protocol usage	Protocol misuse
Impacted service	Broker availability
Impacted function	Message dissemination

Table 4.9: Dataset coverage characteristics

Aspect	Description
All possible attacks covered	No
Attack diversity	High
Realism	High
Protocol-consistent execution	Yes
Suitability for ML-based IDS	Yes

device behavior can change traffic patterns. These factors may not be fully captured by MQTTEEB-D. This is a known limitation.

At the same time, the current situation must be considered. Public MQTT intrusion datasets with the same feature definitions are very few. Because of this, using one trusted dataset is practical. It is also methodologically acceptable.

Universal generalization is not claimed. A reproducible and transparent benchmarking reference is intended. Dataset properties, class imbalance, and threat coverage are clearly documented. In this way, a path is left for future work to follow, extend, or question when other datasets become available.

4.2.3 Data Preprocessing and Feature Engineering

Raw network traffic from IoT systems is messy by nature. It is noisy, mixed, and not ready for machine learning use. Therefore, a structured preprocessing and feature engineering pipeline is applied. Raw MQTT traffic is converted into a consistent format that is ready for modeling.

The emphasis is deliberate. Reproducibility operational realism and preservation of protocol semantics come first. Aggressive dimensionality reduction or artificial balancing takes a back seat.

All preprocessing steps are applied uniformly across the entire dataset before any model training or evaluation takes place. This keeps data handling consistent and ensures that observed performance differences, -which reflect intrinsic learning behavior rather than preprocessing side effects.

Table 4.10: Classification formulation used in this study

Design Choice	Reason
Multi-class classification	Not used
Binary classification	Selected
Class labels	Benign / Malicious
Main objective	Separate malicious from benign traffic
Motivation	Avoid dominance of major attack types

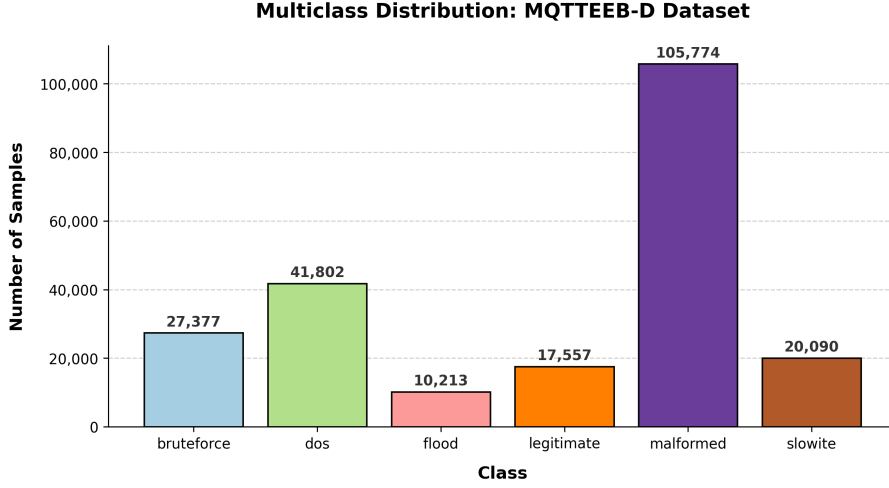


Figure 4.2: Multiclass distribution of attack categories within the MQTTEEB-D dataset.

Dataset Pre-processing and Metadata Documentation

The MQTTEEB-D dataset is distributed in a pre processed form. Protocol level flag fields and categorical labels have already been transformed into numeric representations before release. Two accompanying metadata files describe these transformations.

categorical_processing_metadata.json documents the hex to integer conversions applied to protocol flag fields. In particular TCP flag fields (`tcp_flags`) MQTT connection acknowledgment flags (`mqtt_conack_flags`) MQTT connection flags (`mqtt_conflags`) and MQTT header flags (`mqtt_hdrflags`) were converted from their original hexadecimal format into integer values.

Basically this means the flags arrive ready to use as numeric features. No extra binary encoding steps are required and no missing value imputation is introduced by the experimental pipeline. It is also important to note that no one hot or ordinal encoding was applied to these flag fields during dataset preparation.

label_encoding_metadata.json documents the mapping of multi-class attack labels to integer codes: brute force attacks $\rightarrow$ 0, denial-of-service attacks $\rightarrow$ 1, flooding attacks $\rightarrow$ 2, legitimate traffic $\rightarrow$ 3, malformed packets $\rightarrow$ 4, and slow iterative attacks $\rightarrow$ 5. This pre-existing label encoding enables direct reference to

Table 4.11: Feature preprocessing steps

Step	Operation	Purpose
Timestamp column	Removed	Prevent temporal exploitation
Input features	Traffic-level only	Focus on behavior

Table 4.12: Label transformation scheme

Original Label	Meaning	Binary Label
3	Normal traffic	0
0	Attack	1
1	Attack	1
2	Attack	1
4	Attack	1
5	Attack	1

attack categories while facilitating the subsequent binary reformulation described in Section 5.2.2.

As a result of this pre-processing, the dataset was delivered as a CSV file with all features already in numeric format, eliminating the need for categorical encoding or imputation steps within the experimental pipeline. The in-notebook preprocessing steps are therefore limited to timestamp removal (to prevent data leakage). Creation of the binary classification target, feature standardization for distance-based models, -and class-weight-based balancing for imbalance-aware training.

Inside the experimental notebook, preprocessing is intentionally kept minimal. The idea is not to heavily transform the data but to prepare it in a way that avoids leakage and preserves realistic behavior.

The feature preprocessing steps are shown in Table 4.11. The label mapping used in this study is shown in Table 4.12. When these tables are viewed together, the data preparation strategy can be understood. This strategy supports robust detection and controlled false positive behavior.

The dataset is then split into training and held-out test sets using stratification so that class proportions remain consistent across both partitions.

Feature scaling is applied only where it is actually needed. Distance-sensitive models such as Logistic Regression and balanced Linear SVM use standardized features. Random Forest and LightGBM operate directly on raw features since scaling does not influence tree-based decision rules.

Finally, class imbalance is handled through weighting rather than resampling. Logistic Regression, Linear SVM, and Random Forest use `class_weight="balanced"` so that minority-class samples receive higher importance during training.

Overall, these steps are meant to prepare the data without over-engineering the pipeline and without introducing transformations that could distort the original

Table 4.13: Feature group composition

Feature Group	Examples
Transport layer features	Packet length, TCP flags, timing
MQTT protocol features	Message type, QoS

Table 4.14: Feature group and attack evidence

Feature Group	Captured Behavior
Transport layer features	Flooding activity
Transport layer features	Malformed packets
Transport layer features	Traffic irregularity
MQTT protocol features	Publish/subscribe misuse
MQTT protocol features	Semantic deviations

traffic characteristics.

The SHAP explainability analysis reused the fitted models from these preprocessing steps without introducing additional feature transformations, ensuring that explanations remain grounded in the actual model decision mechanisms.

Figure 4.3 provides a categorical breakdown of the engineered features used in the experimental evaluation.

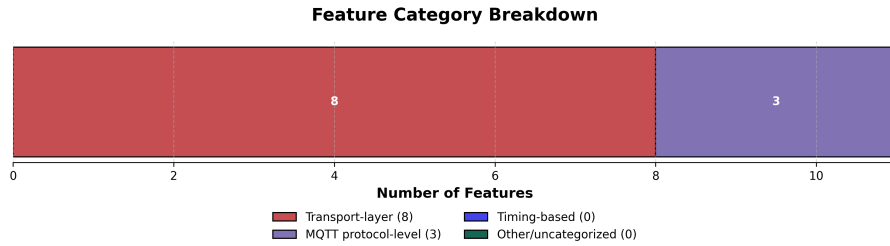


Figure 4.3: Categorical composition of the engineered feature set.

Figure 4.3 illustrates the categorical composition of the engineered feature set used in this study. The majority of features (eight out of eleven) are derived from transport-layer attributes, while three features capture MQTT protocol-level semantics. No standalone timing-only or uncategorized features are included, as timing-related behavior is embedded within transport-level attributes such as packet inter-arrival intervals.

The feature groups used in this study are shown in Table 4.13. The type of attack evidence captured by each group is shown in Table 4.14. When these tables are viewed together, the feature design strategy can be understood. The consistency of this choice is reflected in the explainability results shown in Table 4.15.

Table 4.15: Explainability findings

Observation	Outcome
Dominant feature group	Transport layer
Contribution pattern	Repeated across models
Validation of feature design	Yes

Table 4.16: Binary label encoding scheme for MQTT intrusion detection

Traffic Category	Encoded Label
Normal traffic	0
Attack traffic	1

4.2.4 Label Encoding and Problem Reformulation

Table 4.16 summarizes the encoding approach. Multiple attack types are merged into one malicious class. In this way, sparse labels are reduced and class splitting is limited. At the same time, direct analysis of false positive rates is enabled. This metric is important for IoT security systems. The reasoning behind this reformulation is summarized in Table 4.17. When these tables are viewed together, the deployment-oriented nature of the task becomes clear. This formulation supports reliable detection and interpretable false positive behavior.

Table 4.17: Problem reformulation rationale and operational impact

Aspect	Observation
Deployment focus	Detect benign vs malicious
Class imbalance	Severe
Multi-class effect	Sample fragmentation
Binary formulation effect	Stable learning
Operational priority	Fast and reliable detection
Interpretability	Easier false positive analysis

4.2.5 Handling of Class Imbalance

Table 4.18: Class distribution after preprocessing

Class	Proportion (%)
Attack traffic	~92%
Normal traffic	~8%

The post-processed class distribution is verified in Table 4.18. The strategy used to handle class imbalance is summarized in Table 4.19. When these tables are viewed together, the realism of the evaluation setup becomes clear. The resulting evaluation implications are shown in Table 4.20.

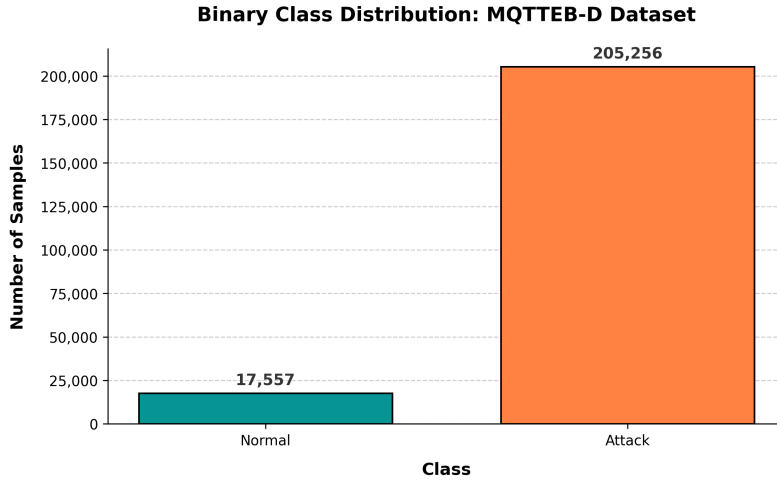


Figure 4.4: Binary class distribution of benign and malicious traffic in the MQTTEB-D dataset.

In Figure 4.4, a strong imbalance between the two classes is observed. Malicious traffic is much higher than benign traffic. This imbalance comes from how the dataset was collected, where attack traffic was emphasized for analysis. The imbalance is not corrected using resampling or reweighting.

By keeping the natural class distribution, false positive behavior is examined under realistic conditions. In such cases, even small false positive rates can create large

Table 4.19: Class imbalance handling strategy

Aspect	Setting
Dataset balance	Heavily imbalanced
Attack proportion	$\sim 92\%$
Resampling used	No
Original distribution	Preserved
Applied across	All experiments

Table 4.20: Implications of preserving class imbalance

Aspect	Outcome
Realism	Maintained
Metric inflation risk	Reduced
Evaluation difficulty	Increased
False positive analysis	Enabled
Deployment relevance	Improved

operational burden. For this reason, false positive rate is treated as a key evaluation metric. The binary distribution in Figure 4.4 therefore provides the basis for the evaluation strategy used in Chapter 5.

4.2.6 Feature Cleaning and Transformation

Feature cleaning focused on a few practical steps. Redundant identifiers were removed. Missing or inconsistent values were resolved. Numerical attributes were normalized where it made sense. Categorical features were converted into numeric representations compatible with the selected machine learning models. All of these transformations were deterministic, and applied consistently across both training and test partitions.

Normalization was used selectively. It mainly serves to stabilize optimization for linear models. Tree based models were evaluated using the same feature representations without introducing model specific preprocessing. This unified handling avoids preprocessing induced bias and keeps comparisons across classifiers fair.

Table 4.21: Feature categorization in the MQTTEEB-D dataset

Feature Category	Description
Transport-layer features	TCP-level statistics and timing characteristics
Protocol-specific features	MQTT control packet and session attributes
Flow-level features	Aggregate statistics over communication flows

Table 4.22: Benchmarking design philosophy

Aspect	Setting
Study focus	Benchmarking
Algorithmic novelty	Not primary goal
Model treatment	Same across all models
Experiment type	Controlled
Bias reduction	Emphasized

Table 4.23: Data splitting and reproducibility controls

Item	Setting
Train portion	70%
Test portion	30%
Sampling method	Stratified
Random seed	42
Used for	Split, initialization, tuning

Table 4.21 shows the main groups of features used in this study. Features from the transport layer and features specific to MQTT are both included. This is done on purpose. In this way, it can be checked whether detection decisions are based more on general network behavior or on MQTT-specific information.

This difference is important for understanding how the models work. It becomes especially useful in the explainability analysis. There, feature importance values are examined. These values are used to show how much each feature group helps in making a decision.

Preprocessing Design Implications

The preprocessing strategy is kept conservative. Aggressive feature removal is not used. Synthetic data balancing is also not applied. Because of this, the natural complexity of real MQTT traffic is preserved. The classification task is made harder. However, the evaluation is made more honest.

Feature representations produced by this pipeline support effective model training. At the same time, meaningful interpretation is still possible. Explainability results therefore reflect real traffic behavior. They are not shaped by preprocessing effects. Taken together, these decisions strengthen benchmarking validity and keep the evaluation close to real operational conditions.

4.3 Implementation and Benchmarking Framework

The overall benchmarking philosophy is summarized in Table 4.22. The data splitting and reproducibility settings are shown in Table 4.23. When these tables are

Table 4.24: Test-set isolation rules

Rule	Applied
Test set used for training	No
Test set used for tuning	No
Test set used for resampling	No
Test set used for augmentation	No
Purpose	Prevent leakage

viewed together, the fairness of the experimental setup becomes clear. The strict test set isolation rules are listed in Table 4.24.

Table 4.23 lists the experimental settings used for all models. These settings are kept the same. In this way, variation that is not related to model behavior is removed. When performance differences are observed, they can be linked to how the models learn. They are not caused by changing experimental conditions. This type of control is necessary for fair and valid comparison between models.

4.3.1 Benchmarking Procedure

The benchmarking procedure is built on one simple idea. All models are tested in the same way. This is done so that the results are fair. After preprocessing, the same training data is given to every model. The same test data is also used for every model. In this way, each model is placed in the same situation.

When tuning is needed, it is done only with the training data. Cross-validation is used for this step. The test data is not touched during tuning. This keeps the test results clean and good. Models are trained one by one as predictions are made one by one. Evaluation scores are also calculated one by one. Because of this, the order of testing is kept the same for all models. No special data changes are made for any single model. No extra preprocessing steps are added. This helps prevent unfair advantage.

Algorithm 1 shows these steps clearly. Tuning, training, and testing are separated into different stages. Training data and test data are kept apart. Data leakage is avoided. In the end, a simple and clear pipeline is created. All models are judged under equal conditions. Because of this, the results can be trusted. Other researchers can repeat the same steps and check the findings.

Input: Preprocessed dataset $\mathcal{D}$, model set $\mathcal{M}$
Output: Performance metrics for each model
Split $\mathcal{D}$ into stratified training set $\mathcal{D}_{train}$ and test set $\mathcal{D}_{test}$
foreach *model* $m \in \mathcal{M}$ **do**
 if *m requires tuning* **then**
 Perform RandomizedSearchCV on $\mathcal{D}_{train}$
 Select best hyperparameter configuration
 end
 Train m on $\mathcal{D}_{train}$
 Evaluate m on $\mathcal{D}_{test}$
 Compute evaluation metrics
end
return *Model-wise performance metrics*
Algorithm 1: Benchmarking Procedure for MQTT Intrusion Detection

4.3.2 Evaluation Metrics and Measurement Discipline

Table 4.25: Evaluation metrics and operational relevance

Metric	Operational Significance
Accuracy	Overall correctness (misleading under imbalance)
Precision	Reliability of attack alerts
Recall	Attack detection coverage
F1-score	Balance between precision and recall
False Positive Rate	Operational cost of false alarms

Reproducibility and Reporting Integrity

The operational role of each evaluation metric is summarized in Table 4.25. The metric computation settings are shown in Table 4.26. When these tables are viewed together, the evaluation logic can be clearly understood. The reproducibility and reporting controls are listed in Table 4.27.

4.3.3 Computational Efficiency and IoT Deployment Feasibility

Detection accuracy alone is not enough. In MQTT based IoT environments the practical viability of an intrusion detection system is tightly coupled to computational efficiency latency behavior and memory footprint. These systems often run under strict resource constraints. Even modest increases in inference time or model complexity can translate into sluggish response and poor real world usability.

Table 4.26: Metric computation settings

Aspect	Setting
Averaging method	Binary
Positive class	Attack traffic
Confusion matrix stored	Yes
Error analysis	Enabled
Link to explainability	Yes

Table 4.27: Reproducibility and reporting controls

Item	Setting
Random seeds	Fixed
Controls applied to	Split, tuning, initialization
Hyperparameter search results	Retained
Final model selection based on	CV on training data
Protocol transparency	Full

So computational feasibility is treated as a core part of the benchmarking framework rather than an afterthought. The goal is to understand not only which model detects best but which model can realistically be deployed.

The relative training-time behavior of the models is summarized in Table 4.28. The inference-time characteristics are shown in Table 4.29 and Table 4.30. When these tables are viewed together, the computational feasibility of the models becomes clear. The memory, scalability, and deployment trade-offs are summarized in Table 4.31 and Table 4.32.

Table 4.28: Training-time characteristics of evaluated models (offline)

Model Type	Relative Training Time	Reason
Linear models	Very low	Simple optimization and few trainable parameters
Random Forest	High	Many trees and additional tuning overhead
LightGBM	Moderate	Efficient boosting implementation under constrained search
Tuned ensembles	Higher than linear	Hyperparameter search adds offline runtime cost

Table 4.29: Inference-time behavior by model type

Model Type	Decision Mechanism	Inference Cost
Linear models	Linear transform over the input feature vector	Minimal
Random Forest	Traversal across multiple decision trees	Moderate
LightGBM	Traversal across boosted decision trees	Moderate
All models	CPU-based evaluation under centralized pipeline	Feasible

Training-Time Characteristics (Offline)

Training is executed offline under a centralized CPU-based workflow, and its cost is treated as an operational constraint due to the need for periodic retraining.

Inference-Time Characteristics

All evaluated models were trained and evaluated in an offline, CPU-based execution environment. During inference, linear models incur minimal computational overhead due to their simple decision functions, while tree-based ensembles require bounded tree traversal.

Table 4.30: Relative computational characteristics of evaluated models

Model	Inference Complexity	Com-	Deployment Suit-	ability
Logistic Regression	Very low		Edge / Gateway	
Linear SVM	Low		Edge / Gateway	
Random Forest	Moderate		Gateway / Broker	
LightGBM	Moderate		Gateway / Broker	

Table 4.31: Memory and scalability properties

Aspect	Observation
Linear models	Store a single weight vector
Ensemble models	Store multiple decision trees
Tree depth	Constrained during tuning
Historical state required	No
Inference type	Stateless
Scalability	High (bounded per-sample computation)

Table 4.32: Deployment-oriented trade-off between detection power and computational cost

Model Group	Detection Power	Computational Cost
Linear models	Lower	Very low
Ensemble models	Higher	Moderate

Latency Considerations Without Hardware Overclaiming

Explicit latency measurements on embedded hardware were not conducted in this study. Instead, feasibility is assessed through relative model complexity and CPU-based inference behavior to avoid hardware-specific claims.

Memory Footprint and Model Scalability

Memory requirements are assessed qualitatively based on model structure and whether inference is stateless.

Deployment-Oriented Interpretation

The deployment trade-off between detection capability and computational cost is summarized in Table 4.32.

By incorporating these constraints into model comparison, conclusions remain deployment-aware rather than accuracy-centric.

Chapter 5

Result Analysis

5.1 Results and Analysis

5.1.1 Performance Evaluation

The main aim of this research was to do a comparative study of Machine Learning Models; hence, this segment is exclusively focused on presenting the outcome in a visually coherent and structured manner. A key point to note is that, in this complete experimental research, a unified benchmarking framework was maintained throughout all the models, which is elaborately explained in Chapter 4. However, rather than solely reporting the aggregated classification scores, the focus is more on a particular model's operational behavioral pattern under severe class imbalance. Additionally, addressing how that pattern functions in the field of MQTT based IoT intrusion detection. Despite training and hyperparameter tuning, the testing data was kept hidden from the model to prevent memorization of traffic patterns. Hence, the outcome of these experiments reflects real-world applications.

Table 5.1: Performance comparison of evaluated models on the MQTTEEB-D test set (training time is measured in seconds; for Random Forest and LightGBM it reflects the total hyperparameter search runtime in the described tuning setup)

Model		Accuracy	Precision	Recall	F1-score	FPR	Training Time (s)
Logistic Regression (Balanced)		0.8699	0.9474	0.9093	0.9279	0.5905	0.1395
Linear SVM (Balanced)		0.8711	0.9474	0.9106	0.9286	0.5905	0.2624
Random Forest (Tuned)		0.8600	0.9755	0.8698	0.9196	0.2550	119.20
LightGBM (Tuned)		0.8618	0.9801	0.8676	0.9204	0.2058	93.38

Table 5.1 illustrates a comprehensive view of model performance with respect to multiple evaluation metrics. To summarize the linear models, Logistic Regression and Linear SVM achieved high precision, contradictorily exhibiting more false positive

rates, which indicates an obvious tendency to misclassify benign traffic as malicious. Conversely, tree-based models substantially reduce false alarms; for example, LightGBM is seen to achieve the lowest false positive rate while maintaining strong overall detection performance.

Importantly, the two tuned ensemble models are closely matched in overall quality (accuracy and F1-score), but LightGBM provides a more favorable operational profile for MQTT IDS deployment: it produces fewer false alarms (FPR 0.2058 vs 0.2550 for Random Forest) and required less training time in the tuning setup (93.38 s vs 119.20 s). Accordingly, Random Forest is a tough competitor and stands as runner up. Notably LightGBM more effectively fulfills the thesis objective of minimizing and reducing false positive behavior under severe class imbalance. This result established how accuracy-centric evaluation under severe class imbalance is inadequate and emphasis the importance of monitoring false positive rate as a key metric.

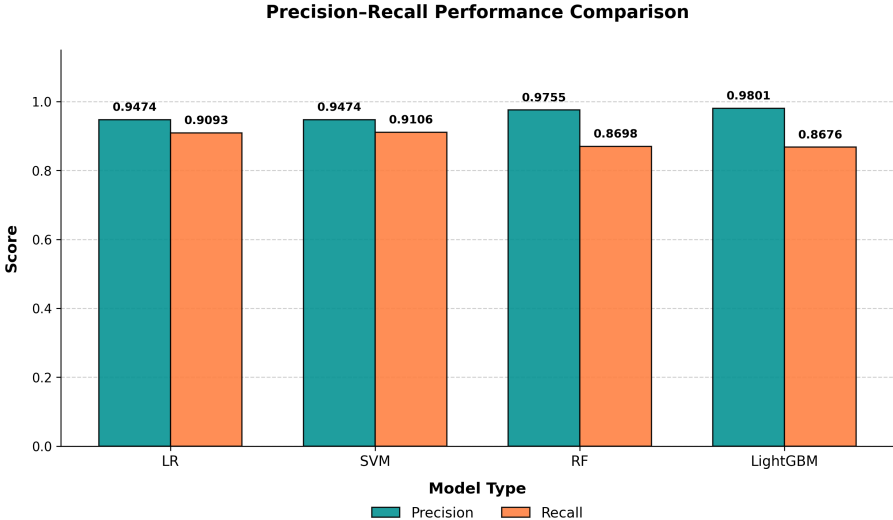


Figure 5.1: Precision-recall comparison of evaluated models on the MQTTEEB-D test set

Figure 5.2 highlights a significant disparity in false positive rate across the evaluated models.

Logistic Regression and Linear SVM, with approximately 0.59 exhibits exceptionally high false positive rates, implying that these models incorrectly flagged more than half of benign MQTT traffic instances as malicious. During operation in IoT environments, such behavior would generate excessive alerts. Which is result the security analysts being overwhelmed, and cause significant increase in response costs. An important finding would be that although these linear models achieve competitive recall, their ability to capture non-linear feature interactions is compromised, and that results in poor discrimination between legitimate and malicious traffic patterns.

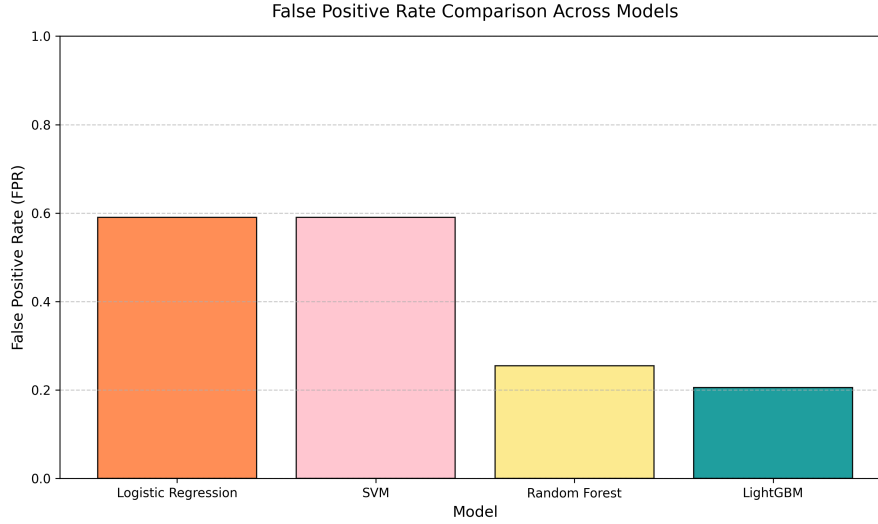


Figure 5.2: Comparative analysis of false positive rates across all evaluated models.

In contrast, tree-based models have significantly enhanced control over false positives. LightGBM achieves the lowest false positive rate (0.2058) compared to Random Forest (0.26). This reduction reflects the ability of tree-based ensembles to model complex, non-linear relationships among protocol-level and transport-layer features. From a deployment perspective, the results shown in Figure 5.2 indicate that false positive behavior, rather than raw detection accuracy, is the dominant factor distinguishing deployable intrusion detection models in real-world MQTT environments.

Accuracy Versus Operational Reliability

Although accuracy is commonly reported in intrusion detection studies, its interpretation is misleading in the presence of highly imbalanced class distributions. In the evaluated dataset, models can achieve moderate to high accuracy by disproportionately favoring the dominant attack class, even when false alarms are frequent. SVM have almost similar accuracy but creates 59%+ False Positive rate. These will create burden to deploy MQTT -based system and raise operational costs. Excessive FPR reduces trust, degrades system operability. This creates an extra hassle in the security operators workflow protocol, which is quite overwhelming. Which is why accuracy alone is insufficient for IoT based environments practical performance. This summarizes, that alone FPR and recall cannot be individually taken as performance metrics, but together. Observing Figure 5.3, the strategic positioning of evaluated models in the recall–false positive rate space. Basically this is shown to visualize the trade-offs of operability between each models. Also, we can see a comprehensive overview between recall and false positive rate across all evaluated models. Looking at the upper bound of the plot, Linear classifiers occupies the space, giving recall importance. Surprisingly, FPR is high even though the recall is good enough. See

Figure 5.3.

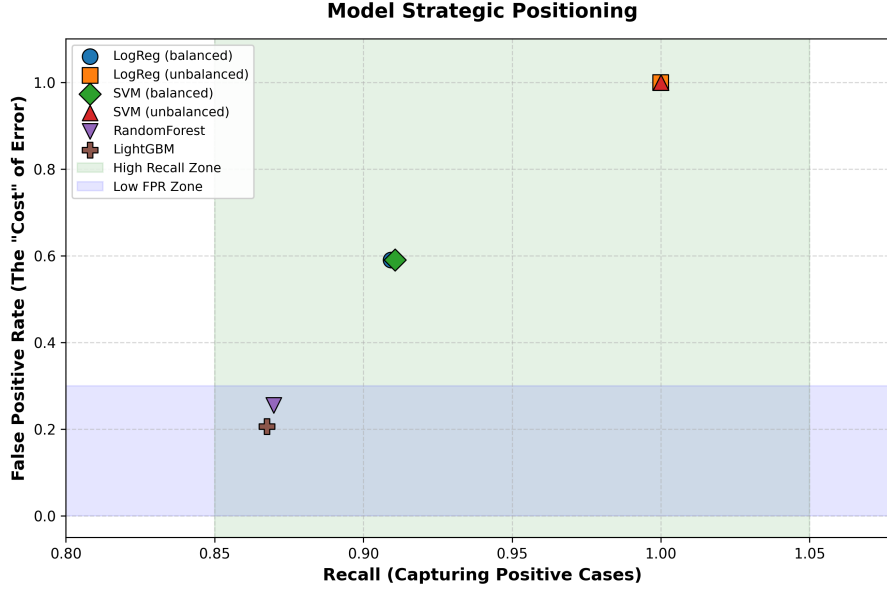


Figure 5.3: Strategic positioning of evaluated models in recall versus false positive rate space.

At a first glance the configuration of results indicates strong potential, demonstrating its attack detection capabilities. However, it comes with a practical bottleneck via false positive results. Which implies that a substantial amount of benign MQTT traffic is getting flagged as malicious. This is not merely a numerical issue but translates directly into higher operational costs and analyst fatigue. Basically, even though linear decision models significantly detect more attacks, they simultaneously produce more false positive results when severe class imbalance enters the picture. At the same time, class imbalance is also important, as it reflects a practical operational environment. Conversely, the performance outcomes of the tree-based models are difficult to disregard: Random Forest and LightGBM withhold low false positive results while receiving strong recall. An unavoidable tradeoff is no model can completely overcome limitations; however, in this scenario, LightGBM proved to be slightly better from an operative point of view. It generated the lowest false positive rate while maintaining the same amount of recall balance. In the field of MQTT protocol research, this finding has a substantial impact. Owning the fact that this reduction in false alarms significantly enhances operational efficiency.

This discovery yields a takeaway that: Identifying the optimal model for MQTT IDS cannot rely on recall alone. Higher recall, if it also generates constant false alarms—that is considered a bad practice. The determining factor is whether the model can withhold a point of equilibrium, identifying attack traffic while keeping false guesses at a lower rate. This study finds that out of all models, LightGBM most consistently satisfies this argument. Figure 5.4. clarifies these off-trade-offs

involved. Making the operating behaviors more concrete via an ROC curve.

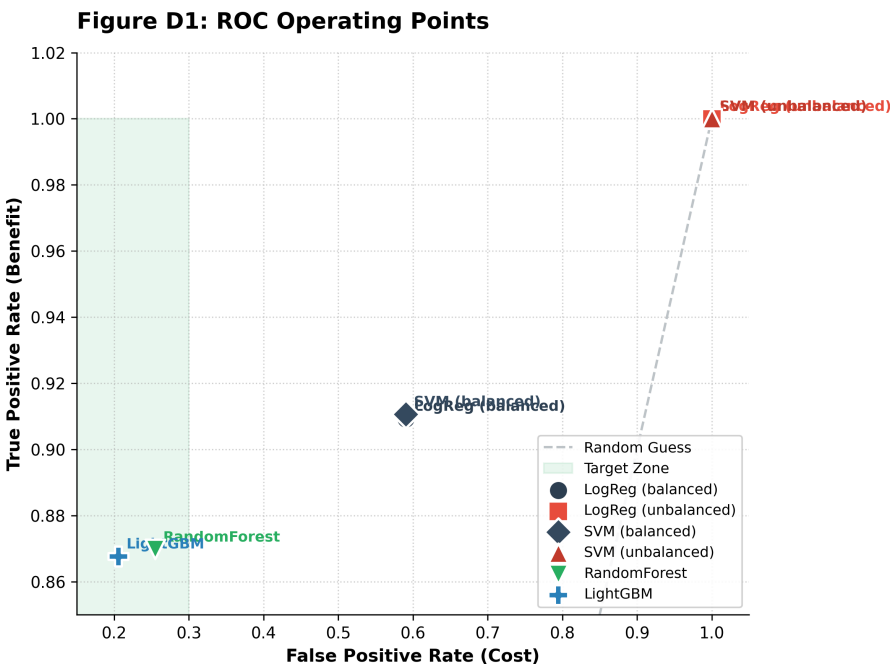


Figure 5.4: ROC operating points corresponding to the selected decision thresholds for each model.

Here the linear model undeniably has higher detection ability. However, this comes at a cost that quickly becomes unmanageable. On the other hand, Random Forest and LightGBM do not aim for the extreme upper corner of the ROC space but instead reside in the lower cost region. These sorts of tree-based models give precedence to manageable FPR in respect to higher accuracy. Viewed in this light, strong ROC performance by itself doesn’t guarantee a model is usable.

The decisive factor is how a model exhibits performance in field conditions rather than if it appears highly effective in theory. Another point of view in this concern is the precision-recall balance, illustrated in Figure 5.5. Up to this point, we have focused on accuracy and false Positive Rate We proceed to examine something more tangible: how credible the generated alerts actually are. As a matter of fact, with higher accuracy Lenier Models also retain stronger recall but still, this is not beneficial. Basically, they do detect many attack behavior but mistakenly flags normal traffic as attacks too, which weakens their credibility.

Under class imbalance, tree-based models show more consistency in performance. In Random Forest and LightGBM, Precision and recall do not compete but rather work simultaneously. They lie within the high reliability zone with balanced outcomes in terms of detection rate and false positive rate. Between the two, LightGBM Marginally outperforms. Environments where MQTT is used as the primary communication protocol between devices, an external barrier of intrusion detection is

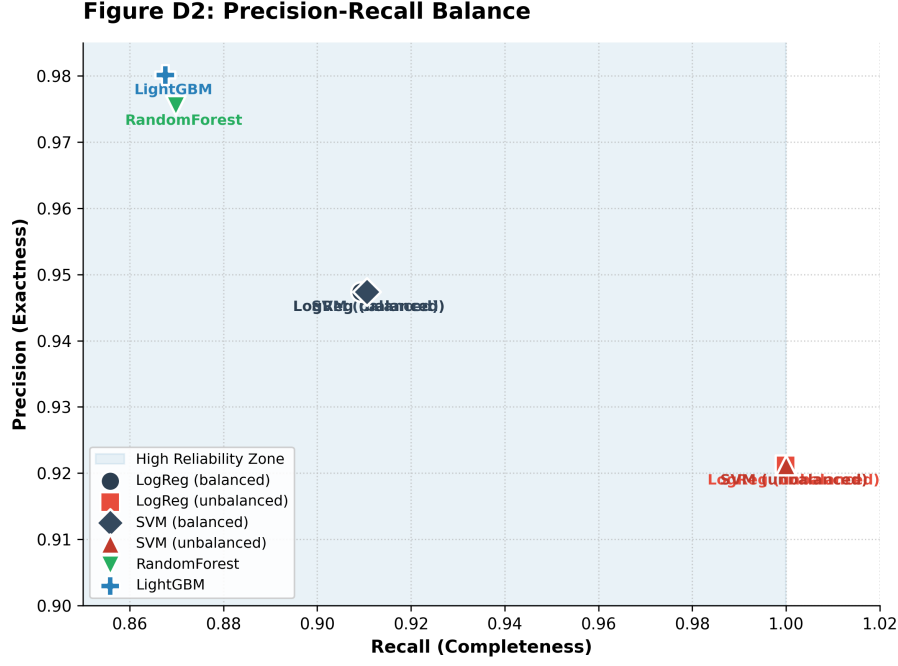


Figure 5.5: Precision versus recall trade-offs across evaluated models.

required. However, excessive false alarms may cause substantial difficulties within the ecosystem. It is merely not a waste of resources but also undermines the workflow of the security analyst and disrupts overall effectiveness.

All together, the claim of the precision-recall pattern from Figure 5.5 is basically affirming the claims of recall-FPR analysis. LightGBM outperforms in terms of reliability and hence should be researched more in regard to the MQTT protocol. Taken together, the precision-recall patterns in Figure 5.5 reinforce what the recall-FPR analysis already suggested. LightGBM does not simply reduce false positives; it also maximizes the practical reliability of the alerts that remain.

Effect of Model Capacity on False Alarm Reduction

In comparison to the linear models, which we considered as baseline models, the tree-based model outperforms them based on the reliability of correct attack detection. It does not only aim to detect threats but also detects them precisely and rarely falsely flags benign traffic. Random Forest brings the false positive rate down to 25.50%, further reduced by LightGBM at 20.58%. This signals meaningful improvement in how benign and attack traffic is classified in MQTT. These numbers represent the most favorable trade-off observed across all evaluated models.

The implicit distributed patterns in MQTT traffic can be complex to work with. In some cases, linear models characterize these as noise and Gradient-boosted decision trees appear better equipped in this case; they form a structure out of it. This observation leads to a broader implication: for controlling false alarms, an analyst

also must consider model structure based on the environment.

While linear classifiers offer simplicity and very low training time, their elevated false positive rates render them unsuitable for realistic MQTT intrusion detection. Among the tuned ensembles, Random Forest and LightGBM deliver competitive accuracy and F1-score, but LightGBM provides the best trade-off by reducing false positives more aggressively while also training faster than Random Forest in the given tuning setup, justifying its selection for subsequent behavioral and explainability analysis.

5.1.2 Analysis of Model Behavior and Design Trade-offs

While established comparative performance

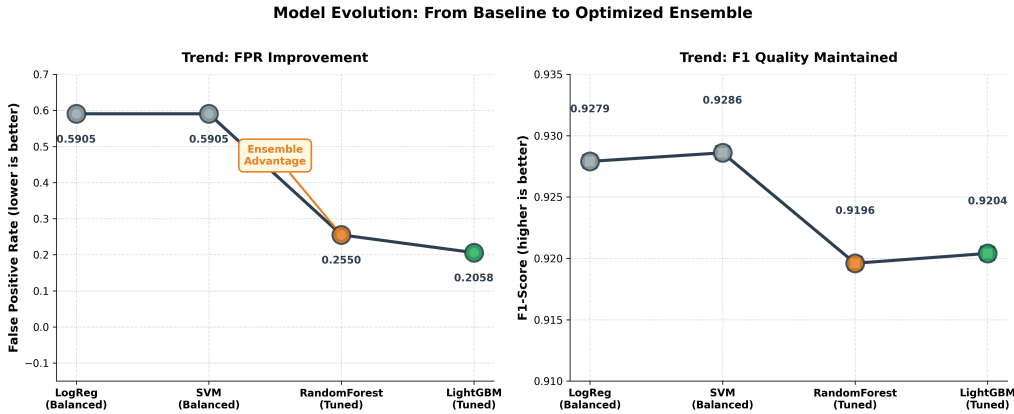


Figure 5.6: Evolution of model performance from linear baselines to ensemble-based classifiers, highlighting changes in false positive rate (left) and F1-score (right).

Upto this point inSection 5.1.1- established comparative performance outcomes, and so this segment is to provide and discuss justification for such behavior. Detection behavior heavily relies on the model architecture, decision boundaries, and feature interaction, especially under imbalanced classes of MQTT traffic.

In the graphs of Figure 5.6, it is shown how models perform in terms of false positive rate (left) and F1-score (right), depicting behavioral differences between linear baseline and ensemble classifiers. The 0.59 percentile for the false positive rate is alarmingly concerning, which is the case for Linear baseline models. This means that out of 100 detected attacks, 59 of them can be falsely accused to be attack types. This is immensely inefficient in terms of security. Later, Random Forest brought down the percentage to 0.21 and was further pushed down by LightGBM. Here, a point to be noted is that the reduction is a false alarm, which does not indicate that attack detection quality has dropped in any way or form. The F1 value remained stable throughout all the models, with a slight decrease in tree-based models in comparison to the linear model. Under this interpretation, optimal

selection never only depends on the numerical threshold. It has to reflect better decision boundaries—boundaries that do not overlap the classification of benign and malicious MQTT traffic. This distinction is important to determine structural capability over post-hoc calibration.

Linear Models: High Recall at the Cost of Operational Noise

As baseline models for this experiment, Logistic Regression and Linear SVM perform quite similarly: High accuracy, high recall, and excessively high false positive recall as well. Which means these models work like a very aggressive guard; any sign of disfigurement in the pattern and the traffic is flagged as an attack. But irregular behavior in patterns is a natural tendency of how traffic works in real life. Hence, while the deployed Linear model does identify attack type, it simultaneously falsely accuses normal traffic of being malicious. So while attack coverage is maximized, practical usability takes a hit, causing operational noise.

Tree-Based Ensembles: Capturing Non-Linear Feature Interactions

Tree-based models had fundamentally different detection behavior. Here, the data is aggregated in multiple decision trees, and every tree is separately trained. This result is a substantial reduction in false alarms. LightGBM takes this one step further. Through gradient boosting, trees are built sequentially, with each iteration focusing on correcting the mistakes of the previous one.

Bias–Variance Considerations in Intrusion Detection

Due to the restrictive functional form, Linear Models have higher bias and variance. Which means it has difficulty in understanding the true pattern of the dataset and misinterprets it. Ensemble models expand representational capacity and hence have a lower bias-variance value. Reason being Random Forest averages the variance, and LightGBM uses gradient boosting. This sort of methodology is absent in the linear model, hence the high bias and variance.

Implications for MQTT-Based IDS Design

An MQTT protocol-based environment requires an external layer of security. Considering the set of models, it is not wise to determine a complex model; in that case, deployability will be compromised in terms of resource and device capacity. On the other hand, merely a simple model will also lack efficiency if the detection capability is inadequate. Ultimately the simplicity or the complexity of a model is not the central consideration; its effectiveness is. Which is why it makes sense to carry it forward into explainability analysis.

Figure 5.7: Comparative analysis of Random Forest and LightGBM illustrating false positive rate reduction alongside comparable F1-score performance and training time.

Linear models theoretically appear promising but also flood the system with false alarms, which is a huge practical problem; hence, they are found to be inadequate. Whereas tree-based models address the issue with a manageable false positive rate. Now, to compare between the two tree-based models, both has alsmot simital F1 score of 0.92. But LightGBM generates 5% fewer false alarms, which is of considerable importance in cybersecurity. Furthermore, real-world hacking data is nonlinear and has hidden class imbalance, where Random Forest uses variance averaging, and LightGBM adapts with its gradient boosting capabilities. As a final point, LightGBM notably takes less training time than Random Forest, making it the optimal choice for further research in intrusion detection for an MQTT-based IoT environment.

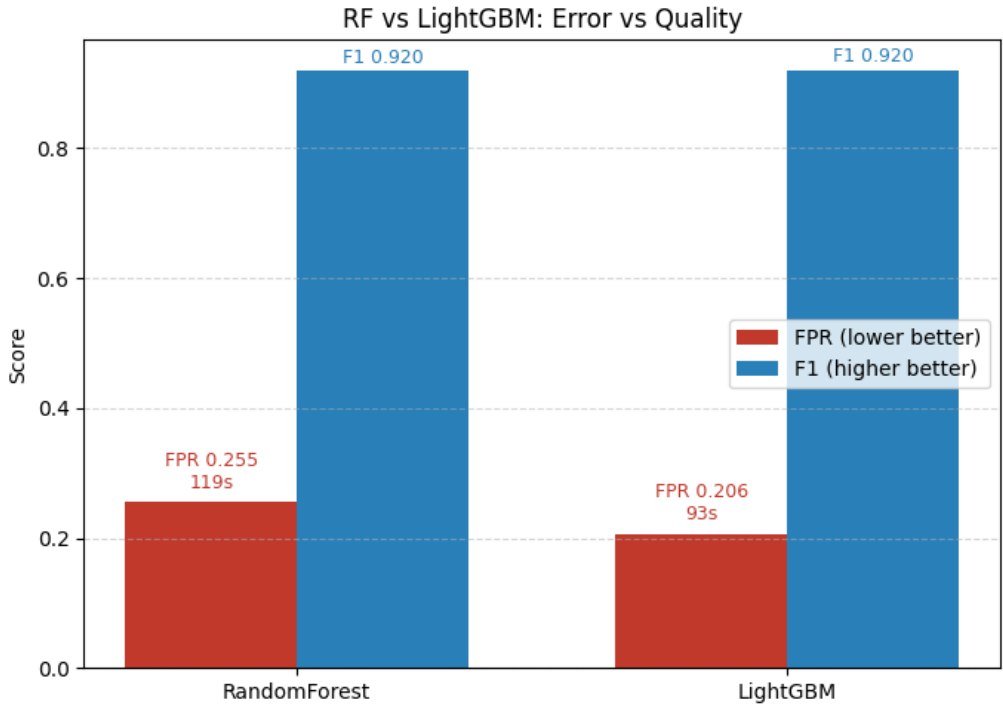


Figure 5.7: Comparative analysis of Random Forest and LightGBM illustrating false positive rate reduction alongside comparable F1-score performance and training time.

Figure 5.7 gives a tighter look at how the two ensemble models separate when error reduction detection quality and computational cost are considered together. Both Random Forest and LightGBM reach almost identical F1-scores at around 0.92 which suggests that overall classification quality is largely comparable. But LightGBM repeatedly produces a lower false positive rate cutting false alarms by roughly five percentage points relative to Random Forest.

From an operational standpoint that gap is not trivial. In MQTT based IoT environments even small drops in false positive rate translate into noticeably fewer alerts fewer interruptions and lower analyst fatigue. LightGBM also delivers this improvement while requiring less total training time under the documented tuning configuration which further strengthens its case as a deployment conscious ensemble model.

What this comparison really shows is that LightGBM does not just edge out Random Forest in isolation. It offers a cleaner balance between detection reliability and computational overhead which is why it is selected as the primary model for subsequent explainability analysis.

Given LightGBM's stronger false positive control without any degradation in overall detection quality the next section turns to why this model behaves differently by examining its feature level decision mechanisms using SHAP.

5.1.3 Explainability and Feature Contribution Analysis

Up until this point, models were evaluated and compared based on the numerical report they were generating. Now it is time to find out the reasoning behind a particle's behavior in a model to confirm it's ground in intrusion detection. Basically, in this segment, Shapely Additive explanation - SHAP is used for feature-level conditions. The study focuses on global feature importance and localized interaction to determine the reasoning.

Previously, we have established that LightGBM is the optimal model for MQTT traffic data. Therefore, further research regarding implementing a Explainable Ai algorithm in a Machine Learning model will be conducted on LightGBM.

Figure 5.8 makes it easier to overview the summary plot of SHAP's workflow, showcasing the significant features that actually impact the model's decision. Transport layer attributes like `tcp_len` and `tcp_time_delta` dominate the timing behavior and payload size patterns. This is what significantly helps the model to separate benign from malicious MQTT traffic.

Point to be noted is that SHAP values demonstrate both positive and negative impact on the feature range, making it non-linear and difficult for a linear model to adapt. SHAP analysis also establishes that the model does not work depending on arbitrary correlation of events, rather trains itself on hacking through features like Transport Layer Timing and Payload characteristics

Overall the explainability results confirm that LightGBM bases its decisions on protocol relevant and interpretable features rather than arbitrary correlations. Transport layer timing and payload characteristics consistently emerge as the dominant

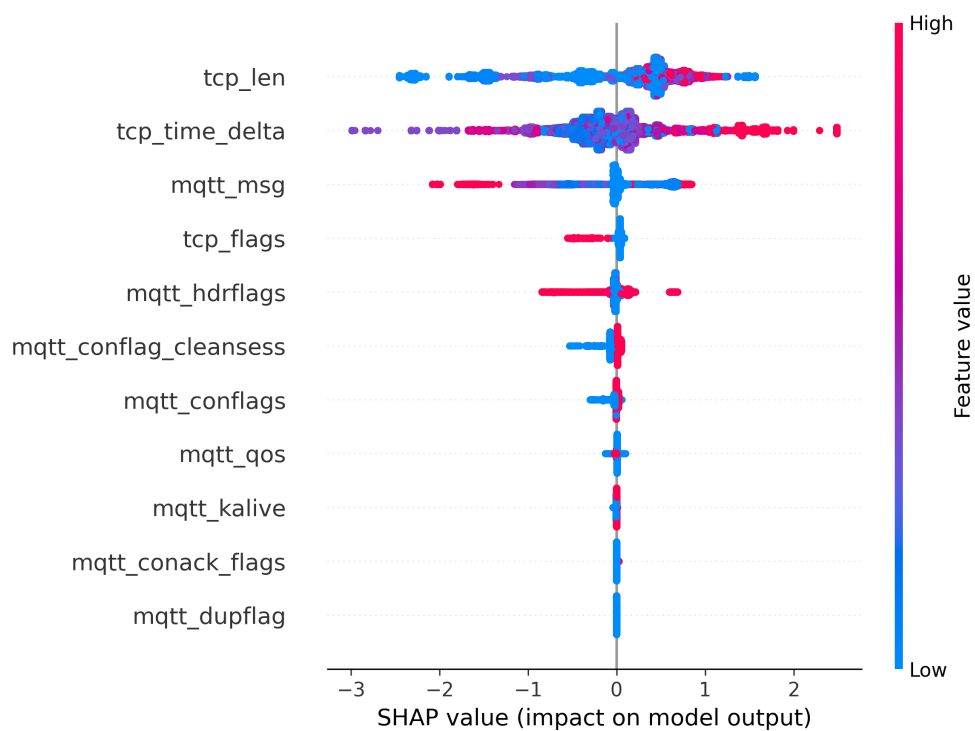


Figure 5.8: SHAP summary plot for the LightGBM model

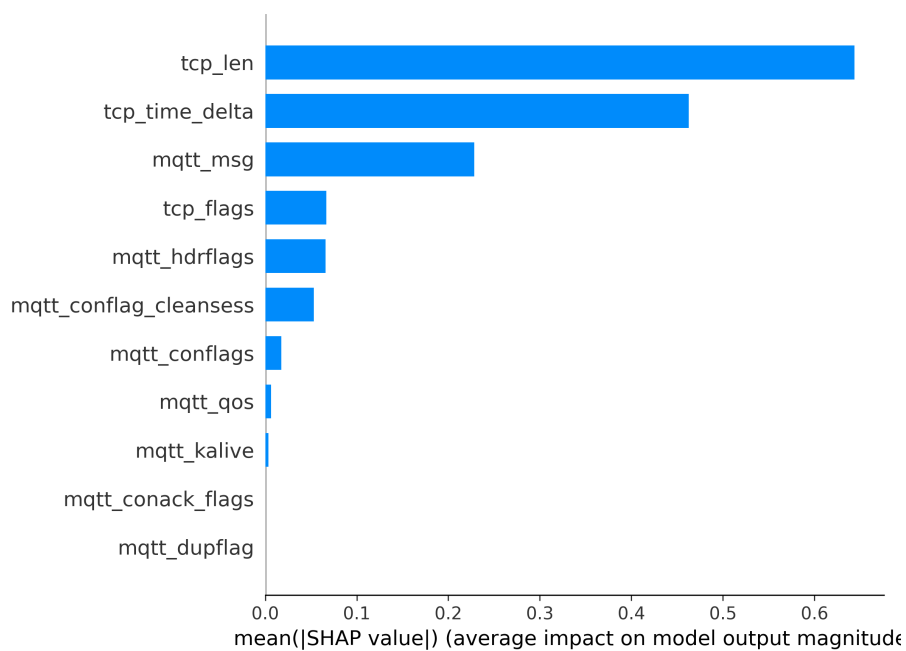


Figure 5.9: Mean absolute SHAP values for top features

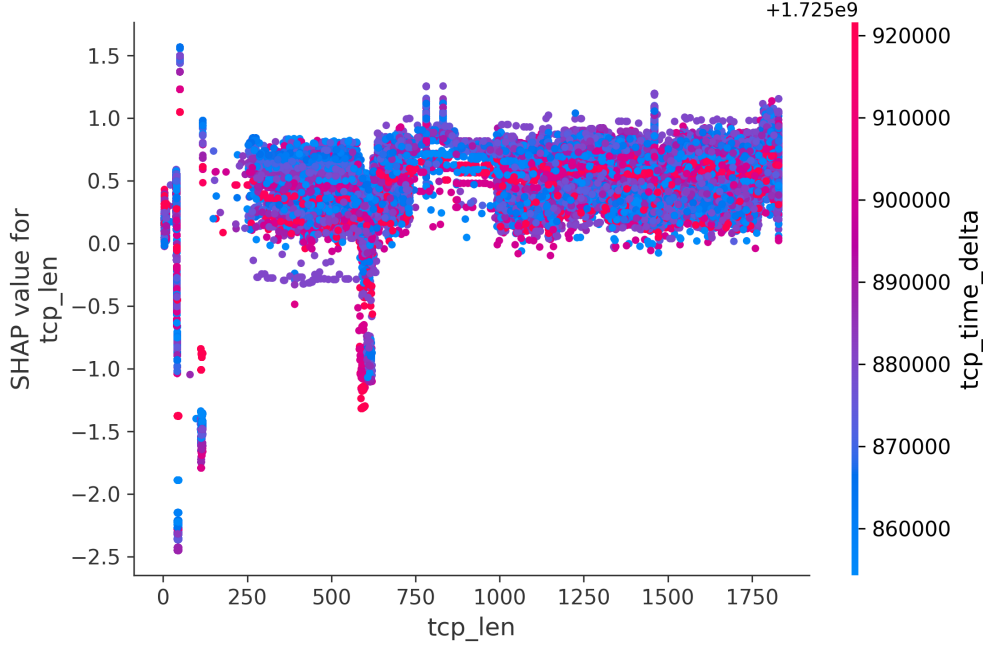


Figure 5.10: SHAP dependence plot for `tcp_len` colored by `tcp_time_delta`

drivers of classification while application layer attributes add contextual refinement. In practical terms this supports both the model choice and the feature engineering strategy used in this study.

To place the LightGBM explainability results in context additional SHAP based feature importance analyses were performed for the remaining evaluated models. These analyses are not meant to provide deep interpretability for every classifier. Instead they serve a comparative purpose showing how different model families rely on protocol and transport level features when making intrusion detection decisions. Figures 5.11 to 5.13 summarizes the mean absolute SHAP values for Logistic Regression balanced SVM and Random Forest respectively. Looking across these figures a clear pattern emerges. The SHAP distributions expose fundamental differences in how linear margin based and ensemble models actually make use of the available features for intrusion detection.

Due to its linear formulation Logistic Regression and Linear SVM models feature a concentration between `mqtt_msg`, `tcp_len`, and `tcp_flags` only. Their model framework relies on an additive formulation, and it cannot capture the higher-order interaction between protocol semantics and transport layer behavior. On top of that, these models ignores timing related dynamic traffic behavior like `tcp_time_delta`, only focusing on static protocol attributes, such as QoS and `message_type`. As a result, due to its static bias, it cannot differentiate between benign irregularity and malicious activity.

As for Random Forest, it shows a shift from this pattern by giving importance to

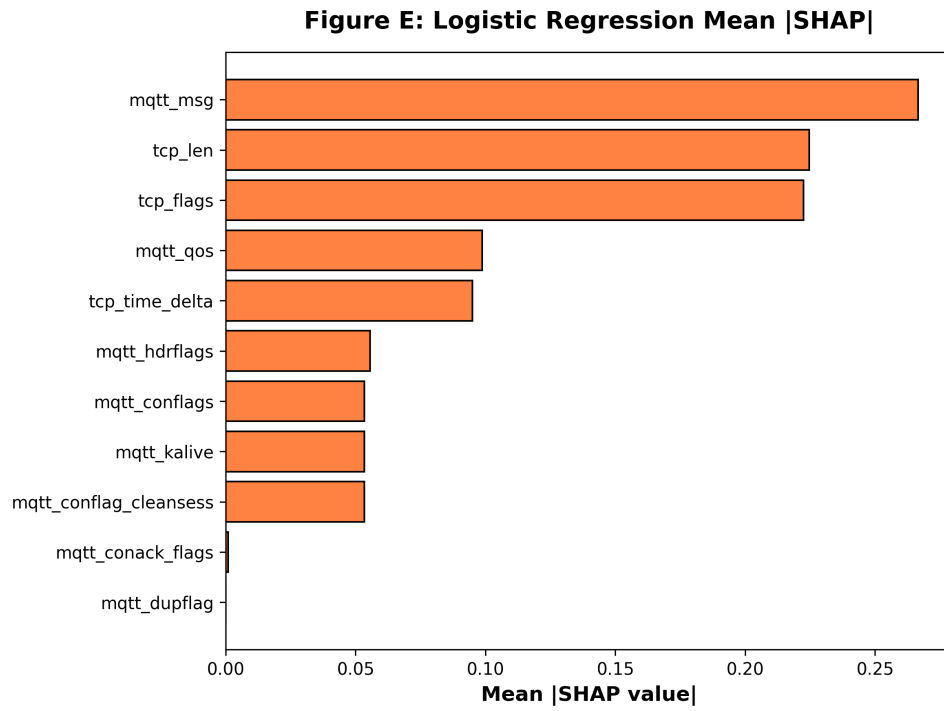


Figure 5.11: Logistic Regression Mean |SHAP|

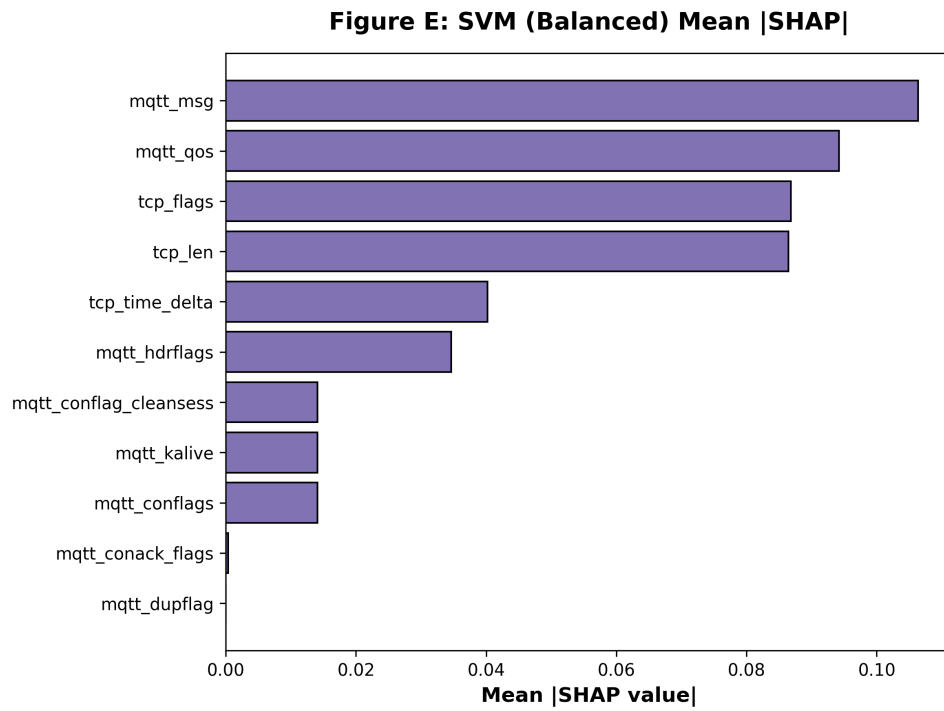


Figure 5.12: SVM (Balanced) Mean |SHAP|

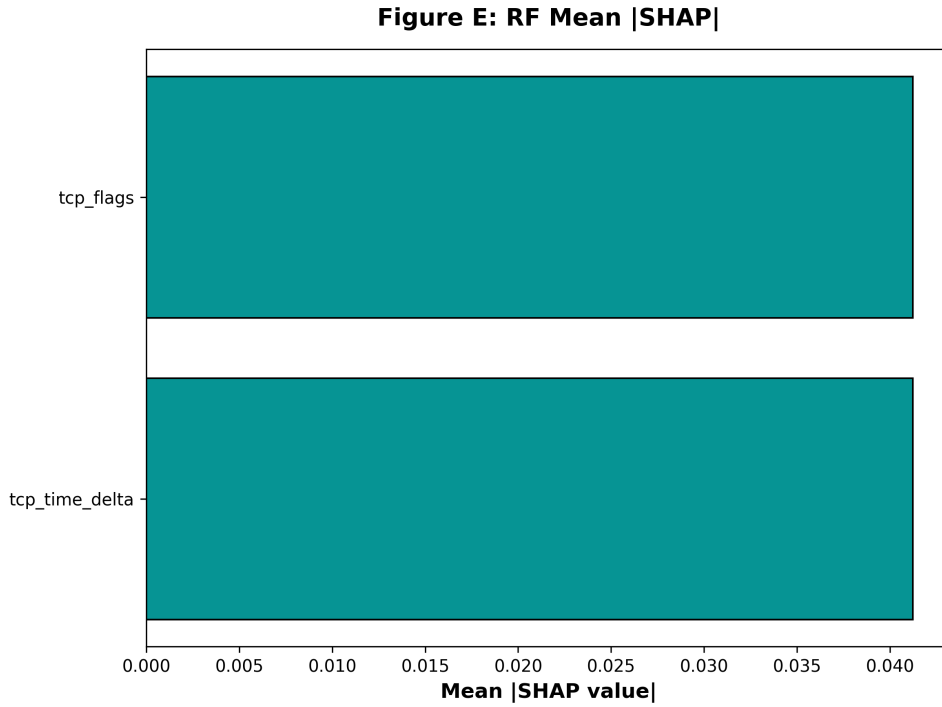


Figure 5.13: Random Forest Mean |SHAP|

transport layer dynamics like `tcp_time_delta`, yet the contribution dept remains shallow. Feature interactions are fragmented and conducted locally, making it difficult to interpret at a global level. LightGBM moves ahead in this direction by maintaining correlation between transport and protocol-level attributes. The model spreads weight evenly and presents coherent explanations of a particular detection. Hence, the contribution of LightGBM, along with explainability, is not only confined to detecting attacks with minimum false alarms but also creates a clarified report for the security analyst to understand how and why an attack is being flagged. After the comparative SHAP application, LightGBM shows consistent performance.

5.1.4 Critical Analysis

Figure 5.9 gives a compressed view of feature importance based on mean absolute SHAP values. Showing clear dominance of `tcp_len` and `tcp_time_delta`, reinforcing the SHAP summary plot overview. Despite MQTT-specific attributes (message type, header flags) being important, these are still considered as secondary influences in this scenario. An attack cannot be seamlessly detected only through application-layer semantics (MQTT features). The interaction between transport-layer timing and payload characteristics is one of the main factors and is more decisive when it comes to identifying anomalous behavior inside MQTT sessions.

Figure 5.10 shifts the focus to plotting packet length `tcp_len` against its SHAP

contribution. This basically means that the payload size can sometimes be benign or malicious, which mainly depends on the temporal context (timing). These sorts of conditional dependencies are difficult for Linear models to train on, whereas tree-based models are more adaptive to them, especially LightGBM, showing utmost consistency.

Here, the main constraint is that despite the promising outcome of ensemble models regarding reducing false alarms while simultaneously detecting attack behavior, their performance effectiveness still greatly depends on the feature quality and the dataset used. MQTT attach is not an isolated attribute but rather created in the protocol layer, coordinating multiple patterns. Taken together, the interaction patterns visible in these plots explain why LightGBM outperforms linear approaches.

5.2 Summary of Findings

This study is a comprehensive comparative study of Machine Learning Models in the field of MQTT-based IoT environments. Its motive was not to propose a new groundbreaking model but to work with existing models and showcase their effectiveness based on the MQTT protocol constraints. Focusing on operational realism and interpretability, despite having a class imbalance, is this study's benchmarking attribute for future researchers.

Detection Performance Under Realistic Imbalance

Under a realistic imbalance Linear classifiers such as Logistic Regression and Linear Support Vector Machine yields high accuracy and recall, defining that they can successfully identify an attack. However, these models also have a higher FPR, indicating that many of the detected attacks may or may not be benign. This misinterpretation of traffic makes this model unreliable for real-world deployment. On the contrary, tree-based models also retain satisfactory accuracy and recall but simultaneously reduce the false positive rate. Which means its decision is more precise and reliable in terms of operational effectiveness. Among the evaluated approaches, LightGBM achieves the most favorable balance, explicitly described in Chapter 5.

Model Behavior and Design Trade-offs

The main reason behind the performance gap among these models is the capability of capturing nonlinear interactions. When benign and malicious patterns overlap in the feature space, linear decision boundaries struggle to differentiate between

them. Ensemble-based models overcome this limitation via hierarchical feature interactions.

Simply formulating a model cannot be the sole reason for its deployment in the IDS. It must also have a balance between computational efficiency and representational capacity. The goal is to control false alarms without sacrificing the attack coverage.

Explainability and Feature-Level Insights

SHAP contextualizes a model behavior. In this case, the findings are that LightGBM does not work based on randomized search but rather has a protocol-relevant performance and interpretable features. Transport Layer features dominate the model behavior, while MQTT attributes prove context. Moreover, feature interaction analysis shows that traffic attributes can be marked as safe or an attack depending upon the temporal context. These intricate differences are more prone to being detected by tree-based models, and with the help of SHAP, we establish the logical basis of LightGBM outperforming the rest of the models.

5.2.1 Operational and Methodological Implications

The findings of the paper may prove to be very helpful in the context of MQTT intrusion detection systems. Our evaluation of false positive control, realistic traffic distribution, and support post hoc interpretability makes this study a benchmark for future researchers. Excessive false alarms diminish the accuracy capabilities of a model and make it impossible to deploy. Hence, this study made an important discovery by conducting an imbalance-aware evaluation and later implementing explainable AI in the optimal model.

5.2.2 Research Contributions

This research demonstrates that MQTT requires a model that can balance the false alarm and still generate a satisfactory result in intrusion detection. The model has to be adaptive in terms of realistic imbalance and non-linear traffic patterns and should also be reproducible for higher efficiency. And in this research work, the obtained result indicates that LightGBM outperforms any other conventional model. This outcome addresses many gaps in existing research where it's only depended on the accuracy level, overlooking the deployability of the model. Last but not least, the findings of this paper are not only about the ML models; they enlighten facts about MQTT and how it should be approached in terms of cybersecurity.

Research / reporting element	This thesis	Typical status in prior studies	Evidence in this thesis
Real-world MQTT dataset used (MQTTEEB-D, 2025)	✓	△	Dataset and real-time traffic context are explicitly framed as MQTTEEB-D real-world MQTT experiments.
Binary mapping from multi-attack labels (Normal vs Attack)	✓	△	Multi-class attack labels mapped to binary intrusion detection objective.
Severe class imbalance preserved for realism (no test-set resampling)	✓	✗	Explicit statement: experiments use processed dataset without resampling unless stated, preserving natural distribution.
Stratified train–test split for fair evaluation	✓	△	Stratified split is part of the locked experimental design (and reflected in your evaluation protocol).
Operational metric focus: FPR treated as a first-class IDS metric	✓	✗	Metrics justification explicitly includes FPR as “critical for IoT”; accuracy becomes secondary.
Imbalance-aware baselines via class-weighting (LR/SVM balanced)	✓	△	Algorithm-level mitigation via <code>class_weight</code> for LR/SVM is explicitly included as the preferred approach.
Tuning restricted only to RF and LightGBM (no leakage, fair baselines)	✓	△	LightGBM tuning explicitly defined (RandomizedSearchCV, stratified CV); tuning is a deliberate, documented design choice.

Table 5.2: Comparative analysis of research elements (Part 1).

Legend:

✓ = explicitly implemented and clearly reported

△ = partially addressed / inconsistently reported (common in literature)

✗ = rarely explicit / often missing (common in literature)

Note: The intent is not to rank papers, but to show reporting completeness and research rigor.

Research / reporting element	This thesis	Typical status in prior studies	Evidence in this thesis
Cross-validation strategy explicitly stated (Stratified 5-fold CV)	✓	△	CV strategy and folds are explicitly documented (StratifiedKFold, 5-fold).
Full hyperparameter search space reported (not just “best params”)	✓	✗	Full LightGBM search space is explicitly written to address examiner criticism about missing search space.
Explainability only for the selected best model (SHAP for LightGBM only)	✓	✗	SHAP is explicitly mandatory and tied to LightGBM design decisions.
Correct explainability depth: SHAP summary + global bar + dependence (interaction)	✓	✗	Your pipeline explicitly allocates SHAP analysis as a separate notebook step and design requirement.
Reproducibility artifacts (documented preprocessing + metadata files)	✓	△	Preprocessing and encoding steps documented with structured metadata files for reproducibility.
Deployment feasibility discussed (efficiency, latency, complexity in IoT context)	✓	△	Explicit paragraph on efficiency/feasibility evaluation and scope boundaries for embedded benchmarking.

Table 5.3: Comparative analysis of research elements and reporting standards (Part 2).

Tables 5.2 and 5.3 depict a comparative overview of research works to date. There is a clear pattern visible in this trend. Many existing works heavily prioritize the accuracy and recall parameters, sometimes even overlooking the false positive rate and its operational efficiency. Emerging from these limitations, this thesis tries to build the bridge between numbers and practicality. It uses the MQTTEEB-D dataset in its imbalanced format to imitate real-world traffic patterns and addresses the false positive rate as the primary evaluation metric.

This thesis is positioned to narrow that gap. It preserves real world class imbalance in the MQTTEEB-D dataset and treats false positive rate as a first class evaluation metric rather than a secondary statistic.

Furthermore, the study enhances performance benchmarking with SHAP-based Ai explainability, using feature directionality, spread, and interaction effects rather than limiting analysis to feature ranking alone. The performance results strongly validates this positioning, with linear baselines exhibiting false positive rates of approximately 0.59, compared to 0.2058 for the selected LightGBM model.

Tables 5.4 and 5.5 provide a comprehensive comparative analysis of research elements and reporting standards, positioned against representative MQTT and IoT intrusion detection studies. The key distinction is that this thesis explicitly foregrounds false positive rate and operational deployability, where most prior works emphasize aggregate accuracy and recall.

The combined effect of these design choices can be summarized by comparing the behavior of a conventional linear intrusion detection system with the proposed LightGBM-based framework.

Study (year)	MQTT focus	Data realism	Task framing	Core method	Explicit FPR emphasis	Explainability	Notes relevant to “what we did better”
Aqachtoul et al. (2025) MQTTTEEB-D dataset	Yes	Real-world testbed	Dataset paper	Dataset contribution	Not reported	NR	Establishes a real-world MQTT benchmark.
Hindy et al. (2021) MQTT case study	Yes	Simulated (realistic) MQTT IoT network testbed	IDS case study	ML-based IDS	NR	NR	Early ML-for-MQTT framing; often limited in operational metric emphasis (FPR not always central).
Vijayan & Sundar (2023) MQTT DL system	Yes	Public dataset(MQTT Kaggle)	IDS (multi-component DL)	Autoencoder + LSTM/DBN	Yes	NR	Deep learning direction for MQTT IDS; interpretability/false-alarm control often not foregrounded.
Zeghida et al. (2025) XMID-MQTT	Yes	NR	Multi-attack classification	ML/DL + explanation	Explicit FPR emphasis (not main metric)	Yes (explainable focus)	Aligns with explainable MQTT IDS; novelty lies in imbalance handling; SHAP/LIME applied mainly to RF.
Jin et al. (2020) SwiftIDS	Not MQTT-specific	High-speed traffic focus	IDS	LightGBM	NR	NR	Demonstrates LightGBM efficiency for high-throughput IDS; not MQTT-specific.
Ahmed et al. (2024) SHAP + ensemble IDS	CIC-IDS2017	NR	IDS	Bagging/boosting + SHAP	NR	SHAP used	Emphasizes operational false alarms; applies SHAP post-hoc for ensemble interpretation.

Table 5.4: Positioning of this thesis relative to representative MQTT and IoT intrusion detection studies (Part 1 of 2).

Study (year)	MQTT focus	Data realism	Task framing	Core method	Explicit FPR emphasis	Explain ability	Notes relevant to “what we did better”
Hermosilla et al. (2025) SHAP vs LIME forensic XAI	Not MQTT-specific	NR	XAI comparison	SHAP/LIME comparison	NR	Yes	SHAP is widely adopted and reliable for forensic interpretation; used as methodological XAI support.
Khraisat et al. (2025) Federated IDS for IoT	IoT general	NR	IDS	Federated learning	NR	NR	Reflects privacy/distributed IDS trends; this thesis targets single-site MQTT accuracy and false-alarm control.
Almadhor et al. (2025) Transformer anomaly detection	IoT general	NR	Anomaly detection / IDS	Large transformer evaluation	NR	NR	Represents cutting-edge DL but with higher overhead; this work favors deployable, interpretable classical ML.
Berhili et al. (2024) IoT IDS survey	IoT general	— Survey	Survey	—	—	—	Uses this as a “why this research matters” anchor.
This thesis (MQTTEED, 2026)	Yes	Real-world MQTT	Binary (Normal vs Attack)	LR/SVM (balanced) + RF/LGBM (tuned)	FPR treated as decisive operational metric	SHAP applied primarily to final LightGBM model	Explicitly preserves severe class imbalance, treats FPR as an operational metric, and uses SHAP summary, bar, and dependence plots to explain feature influence and interactions (not just rankings).

Table 5.5: Positioning of this thesis relative to representative MQTT and IoT intrusion detection studies (Part 2 of 2).

Metric	Baseline (Linear)	Proposed (LightGBM)
FPR	0.5905	0.2058
Accuracy	0.8699	0.8618
Interpretability	Black-box	SHAP explanations

Table 5.6: System-level comparison between baseline linear model and proposed LightGBM-based framework

Above we highlight three system level improvements of the proposed LightGBM based framework relative to traditional linear baselines as summarized in Table 5.6

- **False Positive Reduction:** The framework achieves a 65% reduction in alert noise lowering the false positive rate from 0.5905 to 0.2058. In practical IoT deployments this difference is substantial. Fewer false alarms mean less analyst overload and a system that can actually be operated day to day.
- **Operational Reliability Without Accuracy Overclaiming:** From the through experiments of this paper, the results show that Baseline Model withholds 0.8699 (86.99%) accuracy, while LightGBM withholds 0.8618 (86.18%). This marginal difference can be overlooked given that LightGBM delivers far stronger false positive control. Hence. In terms of practicality under severe class imbalance, LightGBM retails more operational reliability.
- **From “Black-box” to Interpretable AI:** The proposed framework, LightGBM at the final stage of the pipeline, uses SHAP to generate a report of the reasoning behind its work. Basically, this provides a feature-level transparency, allowing the security analyst to monitor the pattern and reasoning behind a derived decision. This feature omits the possibility of accidental correlation and establishes more reliability on the model’s working mechanism.

Overall, this comparison reinforces the core argument of the thesis: under realistic MQTT conditions, tree-based models provide a stronger foundation for reliable intrusion detection in resource-constrained IoT settings, especially with the addition of SHAP.

Table 5.7: Numerical Comparison of IDS Studies

No.	Study	Dataset	Model	Prec.	Recall	F1	FPR	XAI
1	Hindy et al. [7]	MQTT-IoT-IDS2020	RF (best)	0.9968	0.9968	0.9990	NR	X
2	Hindy et al. [7]	MQTT-IoT-IDS2020	DT	0.9992	0.9996	0.9995	NR	X
3	Vijayan & Sundar [8]	MQTT (synth)	LSTM	0.6214	1.0000	0.7683	0.0855	X
4	Anwar et al. [18]	WSN-DS	FL-LSTM	0.9730	0.9820	0.9780	0.0150	X
5	Anwar et al. [18]	CIC-IDS-17	FL-LSTM	0.9610	0.9670	0.9640	0.0240	X
6	Anwar et al. [18]	UNSW-NB15	FL-LSTM	0.9550	0.9620	0.9580	0.0320	X
7	Ahmed et al. [9]	CIC-IDS-17	Bagging+Boost	NR	NR	0.9619	NR	✓SHAP
8	Hermosilla et al. [22]	UNSW-NB15	XGBoost	NR	NR	NR	NR	✓SHAP/ LIME
9	Hermosilla et al. [22]	UNSW-NB15	TabNet	NR	NR	NR	NR	✓SHAP/ LIME
10	Thesis	MQTTTEEB-D (real)	LightGBM	0.9801	0.8676	0.9204	0.2058	✓SHAP
11	Thesis	MQTTTEEB-D (real)	RF	0.9755	0.8698	0.9196	0.2550	X
12	Thesis	MQTTTEEB-D (real)	Log. Reg.	0.9474	0.9093	0.9279	0.5905	X

Table 5.8: Computational Requirements and Lightweight Justification

Study	Model Type	GPU	RAM	Training Time	Lightweight Justification
Hindy et al. (2020)	RF / DT	X	NR	NR	Classical ML
Vijayan & Sundar (2023)	AE + LSTM + DBN	✓	NR	NR	DL-based
Anwar et al. (2025)	FL-LSTM	✓	NR	NR	Distributed DL
Transformer IDS (2025)	Transformer	✓	NR	NR	Heavy DL
This Thesis (2026)	LR / RF / LightGBM	X	NR	✓(CPU)	Tree-based, non-DL

The numerical comparison table is not intended to assert absolute superiority across studies. This study aims to contextualize how intrusion detection systems are evaluated in the literature. All metrics are presented precisely exactly as published, and missing values are explicitly marked as NR. The table focuses on disparities in dataset realism, task framing, and—most importantly—metric emphasis, particularly the frequent omission of false positive rate. Therefore, the comparison is methodological rather than competitive. See Table 5.8 for computational requirements and lightweight justification, and Table 5.7 for numerical performance comparison.

When it comes to real-world deployment, just a high F1-score alone does not guarantee that an intrusion detection system is optimal. Several prior studies reported in Table 5.7 achieve very high F1-scores but do not elaborate on the false positive rate. Regardless of overall detection performance, excessive false alarms can impose significantly high costs on IoT systems and security operators. Keeping that in mind, this thesis demonstrates that models with similar F1-scores can exhibit substantially different false positive behaviors when evaluated under identical conditions. Therefore, this work contributes in exposing the limitations of F1-centric

evaluation and showing that dependence on aggregate detection metrics alone can lead to misleading deployment decisions.

A very important point to be noted is F1-score is computed using precision and recall, and neither of which incorporates information about true negative classifications. In contrast, when calculating the false positive rate, it explicitly depends on the number of true negatives. Consequently, F1-score and false positive rate are mathematically independent metrics. Empirical results in this study confirm this distinction, as shown in Tables 5.7 and 5.8. Models resulting similar in terms of F1 score later perform vastly differently in false positive rate on the same dataset in identical environment. This finding reinforces the significance of reporting false positive behavior explicitly rather than assuming it is captured by the F1-score.

The presence of NR values in Table 5.7 does not weaken the analysis. Instead, it reveals systematic gaps in reporting practices within the IDS literature. Many prior studies do not report false positive rate, explainability mechanisms, or computational feasibility metrics. In contrast, this thesis reports all core evaluation metrics under a fixed and transparent experimental protocol. Therefore, NR values indicate incomplete reporting in prior work rather than missing analysis in this study. Highlighting these omissions is an intended and informative outcome of the comparison.

Explainability analysis of the advanced stage of this research is applied only after identifying models that satisfy deployment constraints. Models which produce an unacceptable level of false positive rate is not suitable for this to begin with, then to apply SHAP on top of it is totally unnecessary. Here, the question remains, why certain models achieve a reduced false alarm rate, and SHAP is used to explain that. The goal is not to rank features independently, but to validate deployment-oriented decision-making by interpreting model behavior in an operational framework. This deployment-first approach is reflected in both Table 5.7, where XAI indicators are shown, and Table 5.8, where computational feasibility is explicitly documented.

RAM and GPU memory utilization are not typically documented in intrusion detection literature. Most IDS studies justify computational feasibility through methodological facts, which includes model class, execution environment, and training or inference time rather than explicit memory profiling. Here the same convention is adapted and documented in Table 5.8. All experiments (of all models) are conducted using CPU-only execution, under identical hardware conditions. Later, the training time is reported comparatively. Additionally, the selected models are not deep learning algorithms and have low computational complexity. As shown in Table 5.8, this

thesis achieves competitive performance using tree-based models on standard CPU hardware rather than deep learning approaches by Vijayan & Sundar (2023), Anwar et al. (2025), and Transformer IDS (2025) require GPU acceleration.

The lightweight claim of a model cannot be only justified via training time. Instead of classifying a model as lightweight, a combination of factors needs to be conjunctively studied. For example, whether it falls under the machine learning algorithm or the deep learning algorithm, then it's computational requirements, and lastly, it's inference mechanisms. Hence, Training time is not considered as an absolute measure of efficiency, but rather a comparative parameter to use across all models. This is a method of applied IDS research where relative feasibility is prioritized over hardware-specific benchmarking. This thesis tries to achieve deployment-conscious performance without requiring specialized hardware, and the comparative analysis in Table 5.8 demonstrates that.

Minor improvements in accuracy do not necessarily cause computational complexity or deployment cost in resource-constrained MQTT-based IoT environments. Many deep learning models require and incorporate GPU acceleration and inference latency, which makes them impractical for real-world gateways.

Hence, here in this thesis deployability and operational reliability is prioritized over marginal gains. The motive is to achieve an acceptable detection performance while maintaining a low false alarm rate with feasible computational requirements. This perspective is clearly illustrated by comparing entries in Table 5.7 and Table 5.8: although deep learning models may achieve higher F1-scores, they remain incompatible for edge IoT devices due to the computational overhead they produce.

Even if some models may show slightly better performance in terms of accuracy, it is not beneficial to opt for that model considering its computational complexity or deployment cost. Many models require GPU acceleration, which does make them more expensive and difficult to deploy in edge devices. Therefore, this thesis has prioritized operational reliability over marginal gains. Hence acceptable detection performance can be achieved while maintaining a manageable false positive rate. Tables 5.7 and 5.8 are visual representations of this argument: that despite higher F1 score Deep Learning models remain unreliable for deployment due to the computational overhead they produce.

This study takes a methodological stance over an algorithmic one. This study shifts the accuracy-centric viewpoint of conventional IDS to a deployment-aware benchmark. Under real-world conditions, this shift also emphasizes the interpretability of a model. This is a beneficial perspective, especially when in applied cybersecurity research, systems are expected to work outside controlled lab settings and hold up

under real operating pressure.

Tables 5.7 and 5.8 make this shift tangible. They show how a deployment aware perspective leads to a more meaningful way of evaluating intrusion detection systems for MQTT based IoT environments

Taken together the results in this chapter confirm that the study objectives have been achieved. The selected machine learning models were evaluated and compared using real world MQTT traffic with clear focus on false positive behavior and performance trade offs. Explainable AI techniques were then used to interpret the behavior of the best performing model which connects the experimental findings back to the original goals of the study.

Chapter 6

Conclusion

6.1 Summary of Contributions

In this thesis, a complete benchmarking framework is introduced for machine learning based intrusion detection in MQTT environments. The focus is placed on realistic deployment conditions, not on complex or novel model designs. The MQTTEEB-D 2025 dataset is used. Synthetic traffic is avoided. Because of this, models are tested under real class imbalance. The main contribution is methodological. Priority is given to deployment awareness instead of only seeking higher accuracy. False positive rate is treated as a main metric. In resource-limited IoT systems, alert fatigue is a real operational problem. The experimental results show a clear pattern. Linear baseline models may report high accuracy. However, many false alarms are produced. This trade-off is hard to accept in practice. Ensemble models provide a better balance. Among them, LightGBM is found to perform best by combining reliable detection with reasonable computational cost. SHAP-based explainability is included as a core part of the study. It is used to support the main goal. Model decisions are shown to be driven by meaningful transport-layer behavior. They are not driven by random correlations. Taken together, these choices form a reproducible baseline. A step is taken toward closing the gap between machine learning results in theory and trustworthy intrusion detection in real IoT systems.

6.2 Boundaries

Even with wide analysis in this study, the findings should be understood within clear boundaries.

- All experiments are based on the MQTTEEB-D 2025 dataset. This dataset reflects one specific testbed setup and a limited set of threats.

- The main task is Normal vs. Attack. This supports strong detection, but detailed separation of attack types is not provided.
- All results are obtained from a fixed held-out test set. Changes in traffic over time and evolving conditions are not modeled.

Explainability analysis with SHAP is used to improve transparency. However, extra computation is required. The results remain descriptive but can be causal. Feature influence on model decisions is shown. Real attack processes are not proven.

These boundaries do not weaken the main contribution of the study. Instead, the conditions where the benchmark is most valid are defined. Clear directions for future work are also indicated.

6.3 Future Work

Based on the benchmark introduced in this thesis, future work should move toward directions that cannot be fully studied using only static datasets or offline experiments-

- The full pipeline should be re-tested when new real-world MQTT datasets become available. In this way, it can be checked whether similar behavior is observed beyond one testbed.
- Power use, memory use, and latency should be measured on real IoT gateway devices. Practical limits for edge deployment can then be defined.
- Online or incremental versions of the models should be developed. Performance under changing traffic and concept drift can then be observed.

Taken together, these directions push the work forward in a practical way. This thesis already provides a grounded starting point for secure MQTT-based IoT systems by focusing on realistic evaluation and transparent analysis. Future extensions can be built on this foundation. A closer link can be created between machine learning research and reliable real-world cyber-physical systems.

Bibliography

- [1] D. Thangavel, X. Ma, A. Valera, and H.-X. Tan, “Performance evaluation of mqtt and coap via a common middleware,” *International Journal of Web Services Research*, vol. 11, no. 3, pp. 1–16, 2014. DOI: 10.4018/ijwsr.2014070101
- [2] A. Singh and A. Gupta, “Security analysis of the mqtt protocol in the internet of things,” *International Journal of Advanced Computer Science and Applications*, vol. 8, no. 6, pp. 183–190, 2017. DOI: 10.14569/IJACSA.2017.080842
- [3] M. Ammar, G. Russello, and B. Crispo, “Internet of things: A survey on the security of iot frameworks,” *IEEE Communications Surveys & Tutorials*, vol. 20, no. 3, pp. 2231–2268, 2018. DOI: 10.1109/COMST.2018.2864901
- [4] H. Fernandes, H. Pedro, and M. Vieira, “On the security of modern internet of things protocols,” in *Proceedings of the 13th ACM Asia Conference on Computer and Communications Security*, 2018, pp. 1–12. DOI: 10.1145/3196494.3196532
- [5] A. Banks and R. Gupta, “Mqtt security: A survey,” *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 5539–5551, 2019. DOI: 10.1109/JIOT.2019.2890621
- [6] D. Jin, Y. Lu, J. Qin, Z. Cheng, and Z. Mao, “Swiftids: Real-time intrusion detection system based on lightgbm and parallel intrusion detection mechanism,” *Computers & Security*, vol. 97, p. 101984, 2020. DOI: 10.1016/j.cose.2020.101984
- [7] H. Hindy, E. Bayne, M. Bures, C. Tachtatzis, and X. Bellekens, “Machine learning based iot intrusion detection system: An mqtt case study,” in *Advances in Intelligent Systems and Computing*. 2021, pp. 73–84. DOI: 10.1007/978-3-030-64758-2_6
- [8] P. M. Vijayan and S. Sundar, “An automated system of intrusion detection by iot-aided mqtt using autoencoder and lstm-based deep belief network,” *PLOS ONE*, vol. 18, no. 10, 2023. DOI: 10.1371/journal.pone.0291872
- [9] U. Ahmed, J. Zheng, A. Almogren, M. T. Sadiq, A. Rehman, and J. Choi, “Hybrid bagging and boosting with shap-based feature selection for intrusion detection,” *Scientific Reports*, vol. 14, 2024. DOI: 10.1038/s41598-024-81151-1

- [10] Z. Alomari, Z. Li, and A. Makanju, "Lightweight machine learning-based ids for iot environments," in *Proceedings of the Cyber Security in Networking Conference (CSNet)*, 2024, pp. 33–37. DOI: 10.1109/CSNet64211.2024.10851737
- [11] M. Berhili, O. Chaieb, and M. Benabdellah, "Intrusion detection systems in iot based on machine learning: A state of the art," *Procedia Computer Science*, vol. 251, pp. 99–107, 2024. DOI: 10.1016/j.procs.2024.11.089
- [12] A. B. Gadewar, R. V. Patil, S. A. Mahajan, and L. V. Patil, "Automated network vulnerability detection in iot: Lightweight ids approaches for enhanced security," in *Proceedings of the Second International Conference on Advanced Computing and Communication Technologies (ICACCTech)*, 2024, pp. 612–618. DOI: 10.1109/ICACCTech65084.2024.00103
- [13] S. Golestani and D. Makaroff, "Exploring unsupervised one-class classifiers for lightweight intrusion detection in iot systems," in *Proceedings of the 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, 2024, pp. 234–238. DOI: 10.1109/DCOSS-IoT61029.2024.00042
- [14] I. Mutambik, "An efficient flow-based anomaly detection system for enhanced security in iot networks," *Sensors*, vol. 24, p. 7408, 2024. DOI: 10.3390/s24227408
- [15] C. Stolz, F. Li, and J. Zhang, "Implementing lightweight intrusion detection system on resource-constrained devices," in *Proceedings of the Cyber Awareness and Research Symposium (CARS)*, 2024, pp. 1–6. DOI: 10.1109/CARS61786.2024.10778716
- [16] C. Xu, X. Du, L. Li, X. Li, and H. Yu, "End-edge collaborative lightweight secure federated learning for anomaly detection of wireless industrial control systems," *IEEE Open Journal of the Industrial Electronics Society*, pp. 1–11, 2024. DOI: 10.1109/OJIES.2024.3370496
- [17] A. Almadhor et al., "Evaluating large transformer models for anomaly detection in resource-constrained iot devices," *Scientific Reports*, vol. 15, 2025. DOI: 10.1038/s41598-025-21826-5
- [18] R. W. Anwar, M. Abrar, A. Salam, and F. Ullah, "Federated learning with lstm for intrusion detection in iot-based wireless sensor networks," *PeerJ Computer Science*, vol. 11, 2025.
- [19] A. Aqachtoul, K. Karam, A. Elamrani, M. Najib, N. Rafalia, and M. Bakhouya, "Mqtteeb-d: A real-world iot cybersecurity dataset for ai-powered threat detection in mqtt networks," *Data in Brief*, vol. 62, p. 111 897, 2025. DOI: 10.1016/j.dib.2025.111897

- [20] Y. Dakhil and M. Çakmak, “Xai-xgboost: An explainable intrusion detection approach for iomt,” *Scientific Reports*, 2025. DOI: 10.1038/s41598-025-07790-0
- [21] I. Fares and M. Abd Elaziz, “Tfkan: Transformer based on kolmogorov–arnold networks for intrusion detection in iot,” *Egyptian Informatics Journal*, 2025. DOI: 10.1016/j.eij.2025.100666
- [22] P. Hermosilla, S. Berríos, and H. Allende-Cid, “Explainable ai for forensic analysis: A comparative study of shap and lime,” *Applied Sciences*, vol. 15, p. 7329, 2025. DOI: 10.3390/app15137329
- [23] A. Karunamurthy, K. Vijayan, P. Kshirsagar, and K. Tan, “An optimal federated learning-based intrusion detection for iot environment,” *Scientific Reports*, vol. 15, 2025. DOI: 10.1038/s41598-025-93501-8
- [24] A. Khraisat, A. Alazab, M. Alazab, A. Obeidat, S. Singh, and T. Jan, “Federated learning for intrusion detection in iot environments,” *Discover Internet of Things*, vol. 5, 2025. DOI: 10.1007/s43926-025-00169-7
- [25] H. Zeghida, M. Boulaiche, R. Chikh, A. Patel, A. Barros, and A. Bamhdi, “Xmid-mqtt: Explaining machine learning-based intrusion detection system for mqtt protocol in iot environment,” *International Journal of Information Security*, vol. 24, 2025. DOI: 10.1007/s10207-025-01036-w